Qt Creator:

Name: Shyam Sundar C

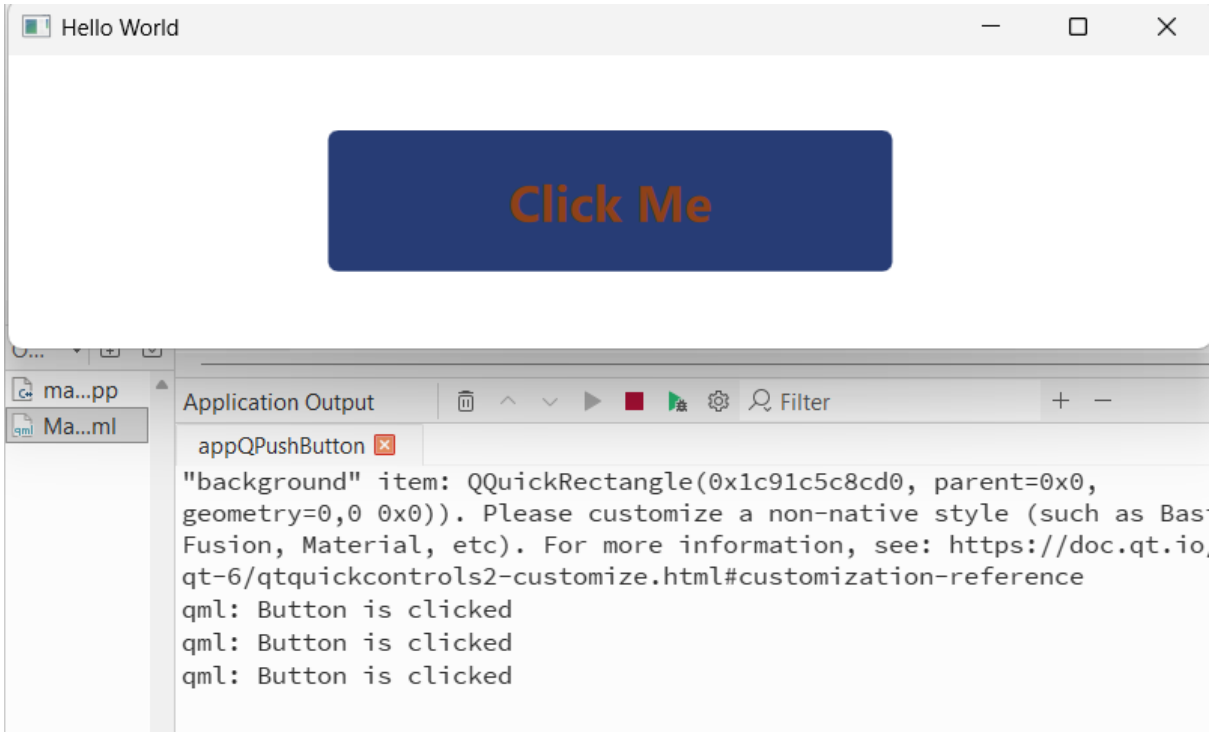
College: PSG College of Technology

1. Button Control:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Button {
        id: myButton
        width: 300
        height: 75
        anchors.centerIn: parent

        Text {
            id: myButtonText
            text: "Click Me"
            color: "#8E4118"
            anchors.fill: parent
            horizontalAlignment: Text.AlignHCenter
            verticalAlignment: Text.AlignVCenter
            elide: Text.ElideRight
            font.pointSize: 20
```

```
        font.bold: true
    }

    onClicked: {
        console.log("Button is clicked")
    }

    background: Rectangle {
        color: myButton.pressed ? "#0097E6" : "#273C75"
        radius: 5
    }
  }
}
```

OUTPUT:

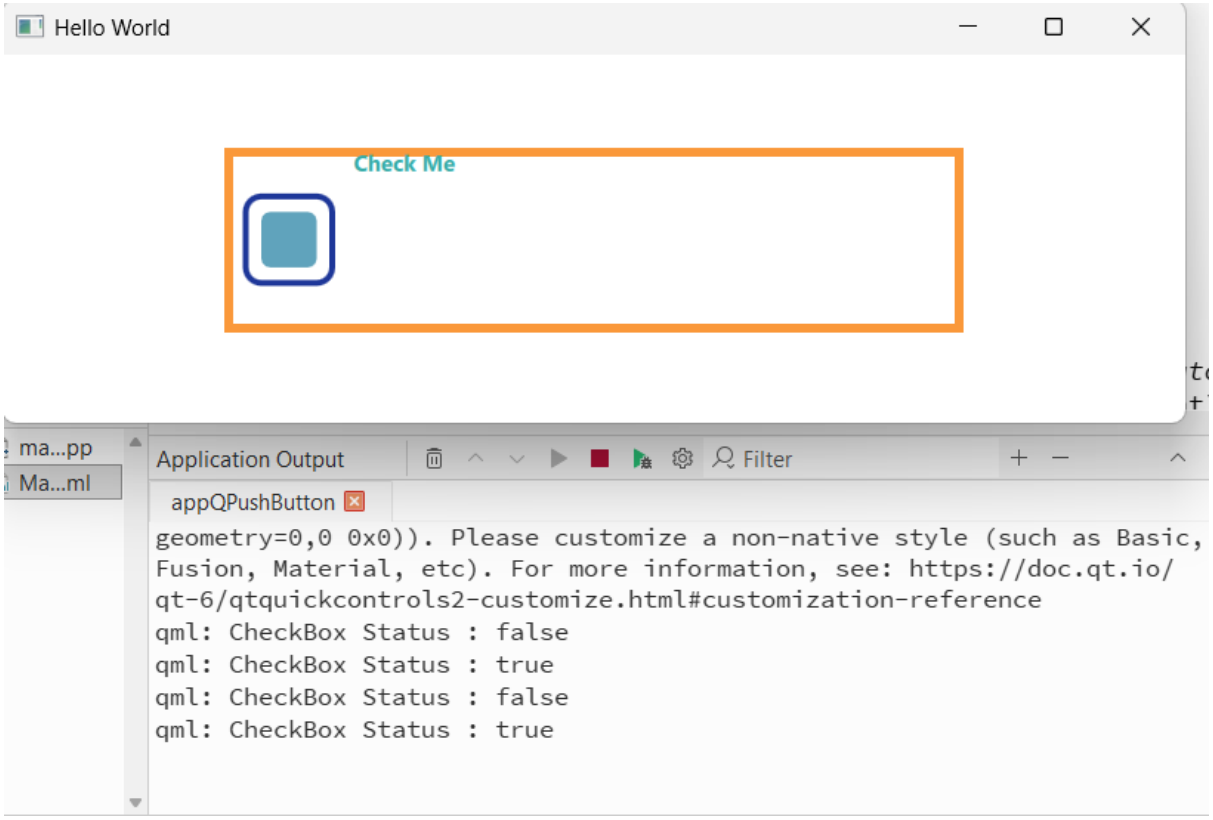


2. CheckBox Control:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    CheckBox {
        id: myCheckBox
        checked: true
        anchors.centerIn: parent
        width: 400
        height: 100

        Text
        {
            id: myCheckBoxText
            text: "Check Me"
            font.bold: true
            color: "#38ada9"
            x: ((myCheckBoxindicator.x + myCheckBoxindicator.width) + 10)
            y: ((mycheckBox.height - myCheckBoxText.height) / 2)
        }

        onCheckedChanged:
        {
            console.log("CheckBox Status : " + checked)
```

```
        }

        background: Rectangle {
            border.width: 5
            border.color: "#fa983a"
        }

        indicator: Rectangle {
            id: myCheckBoxindicator
            implicitWidth: 50
            implicitHeight: 50
            radius: 10
            border.color: "#1e3799"
            border.width: 3
            x: 10
            y: (myCheckBox.height - myCheckBoxindicator.height) / 2

            Rectangle {
                visible: myCheckBox.checked
                color: "#60a3bc"
                border.color: "#60a3bc"
                radius: 5
                anchors.margins: 10
                anchors.fill: parent
            }
        }
    }
}
```

OUTPUT:



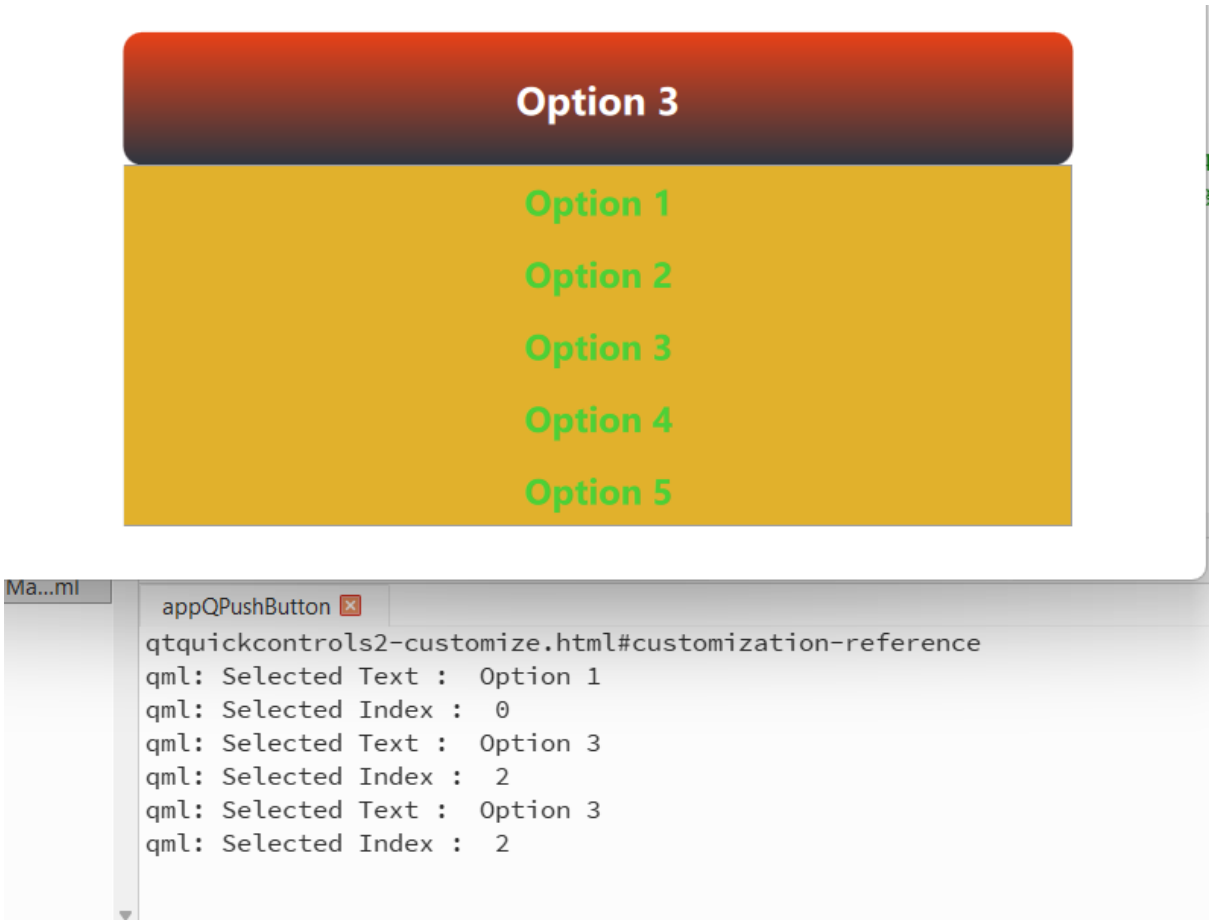
3. ComboBox Control:

```
import QtQuick 2.15

import QtQuick.Window 2.15

import QtQuick.Controls 2.15


Window {

    width: 640

    height: 480

    visible: true

    title: qsTr("Hello World")


    ComboBox

    {

        id: myComboBox

        width: 500
```

```
height: 70
anchors.centerIn: parent
currentIndex: 0
model: myComboBoxListModel

ListModel
{
    id: myComboBoxListModel
    ListElement { key: "Option 1" }
    ListElement { key: "Option 2" }
    ListElement { key: "Option 3" }
    ListElement { key: "Option 4" }
    ListElement { key: "Option 5" }
}

contentItem: Text
{
    color: "#FFFFFF"
    text: myComboBox.displayText
    font.pointSize: 16
    font.bold: true
    verticalAlignment: Text.AlignVCenter
    horizontalAlignment: Text.AlignHCenter
}

delegate: ItemDelegate
{
    width: myComboBox.width
    background: Rectangle
    {
        color: "#e1b12c"
```

```
            radius: 0
        }
        contentItem: Text
        {
            color: "#4cd137"
            text: modelData
            font.pointSize: 14
            font.bold: true
            verticalAlignment: Text.AlignVCenter
            horizontalAlignment: Text.AlignHCenter

        }
    }

    onActivated:
    {
        console.log("Selected Text : ", myComboBox.currentText)
        console.log("Selected Index : ", myComboBox.currentIndex)
    }

    background: Rectangle
    {
        gradient: Gradient
        {
            GradientStop {position: 0.0; color: "#e84118"}
            GradientStop {position: 1.0; color: "#2f3640"}
        }
        radius: 10
    }
  }
}
```

OUTPUT:

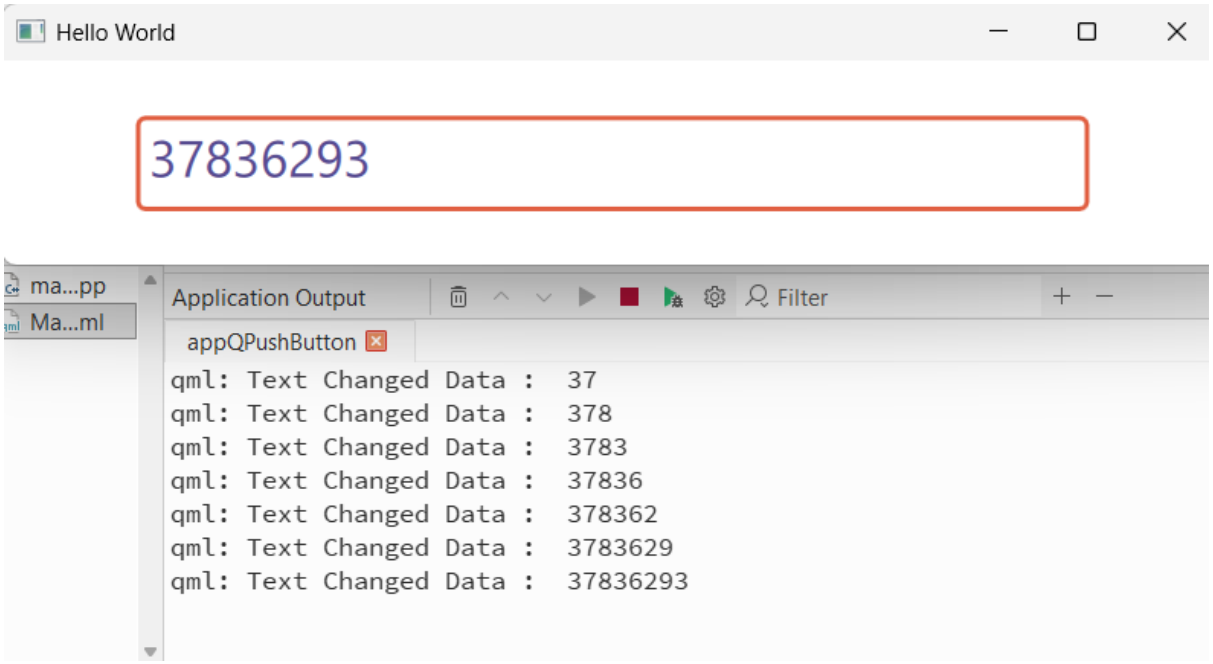


4. TextField Control:

```
import QtQuick 2.15

import QtQuick.Window 2.15

import QtQuick.Controls 2.15


Window {

    width: 640

    height: 480

    visible: true

    title: qsTr("Hello World")


    TextField {
```

```
        id: myTextField
        width: 500
        height: 50
        anchors.centerIn: parent
        placeholderText: "Enter Text Here ..."
        font.pointSize: 20

        //validator: RegExpValidator { regExp: /[0-9]+/ }
        //echoMode: "Password"
        //passwordCharacter: "."

        color: "#574b90"   // Set Your Font Color

        onTextChanged: {
            console.log("Text Changed Data : ", myTextField.text)
        }

        background: Rectangle {
            border.color: myTextField.activeFocus ? "#e15f41" : "#130952"
            border.width: 2
            radius: 5
        }
    }
}
```

OUTPUT:



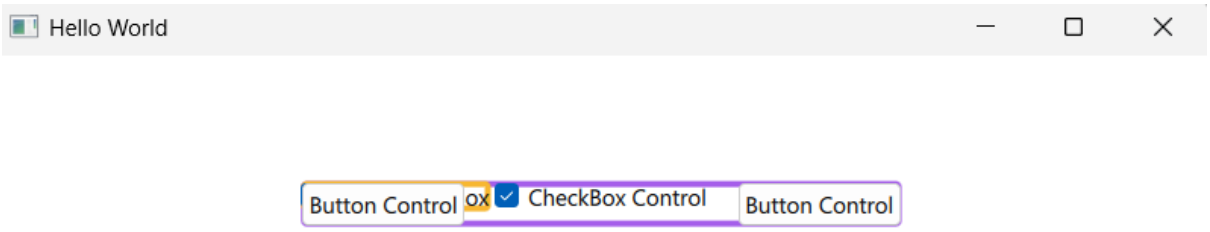
5. GroupBox Control:

```
import QtQuick 2.15

import QtQuick.Window 2.15

import QtQuick.Controls 2.15


Window {

    width: 640

    height: 480

    visible: true

    title: qsTr("Hello World")


    GroupBox {

        anchors.centerIn: parent


        label: CheckBox {

            id: mainCheckBox

            checked: true

            text: qsTr("Main CheckBox")
```

```
    background: Rectangle {
        border.color: "#f7b731"
        border.width: 3
        radius: 3
    }
}

Row {
    anchors.centerIn: parent
    spacing: 15
    enabled: mainCheckBox.checked

    Button {
        id: myButton
        text: qsTr("Button Control")
    }

    CheckBox {
        id: myCheckBox
        text: qsTr("CheckBox Control")
    }

    Button {
        id: myButton_1
        text: qsTr("Button Control")
    }
}

background: Rectangle {
    border.color: "#a55eea"
```

```
        border.width: 3
        radius: 3
    }
  }
}
```

OUTPUT:



6. RadioButton Control:

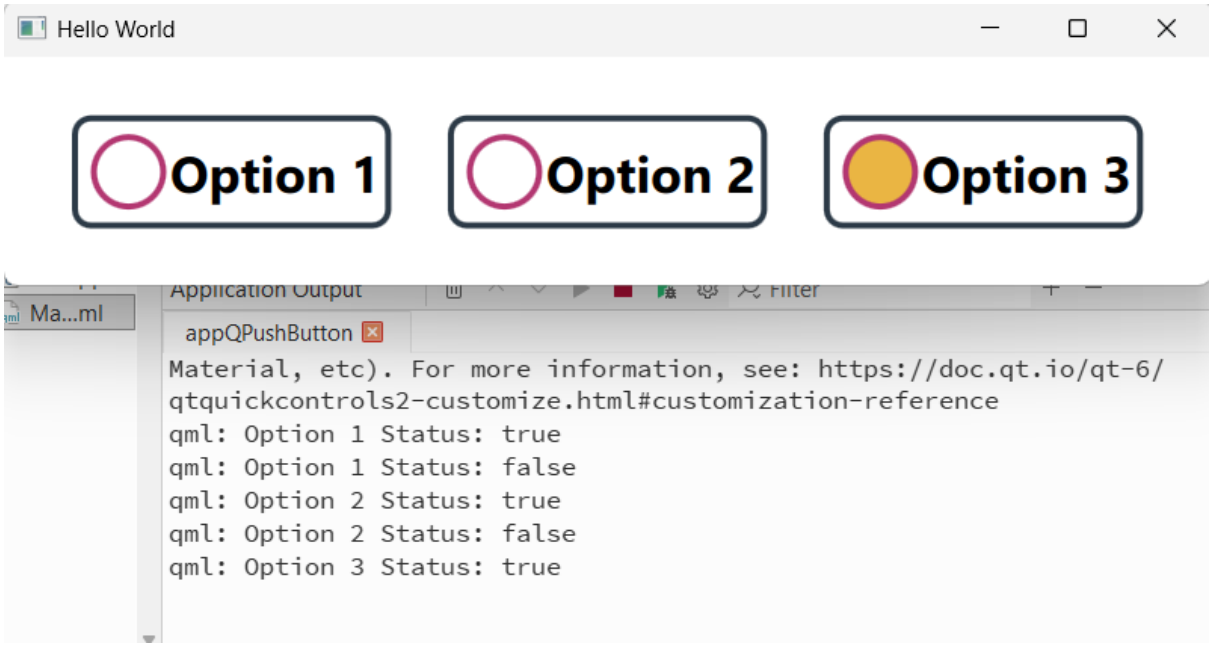
```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15

Window {
  width: 640
  height: 480
  visible: true
  title: qsTr("Hello World")

  ButtonGroup { id: radioGroup }   // makes radios exclusive

  Row {
    anchors.centerIn: parent
    spacing: 30

    // ---------- OPTION 1 ----------
```

```
RadioButton {
    id: myRadioButton_1
    text: "Option 1"
    font.pointSize: 20
    font.bold: true
    height: 60
    ButtonGroup.group: radioGroup

    background: Rectangle {
        border.width: 3
        border.color: "#2C3A47"
        radius: 10
    }

    indicator: Rectangle {
        implicitWidth: 40
        implicitHeight: 40
        radius: width / 2
        x: 10
        y: (parent.height - height) / 2
        color: myRadioButton_1.checked ? "#EAB543" : "transparent"
        border.color: "#B33771"
        border.width: 3
    }

    onCheckedChanged: {
        console.log("Option 1 Status:", checked)
    }
}

// ---------- OPTION 2 ----------
```

```
RadioButton {
    id: myRadioButton_2
    text: "Option 2"
    font.pointSize: 20
    font.bold: true
    height: 60
    ButtonGroup.group: radioGroup

    background: Rectangle {
        border.width: 3
        border.color: "#2C3A47"
        radius: 10
    }

    indicator: Rectangle {
        implicitWidth: 40
        implicitHeight: 40
        radius: width / 2
        x: 10
        y: (parent.height - height) / 2
        color: myRadioButton_2.checked ? "#EAB543" : "transparent"
        border.color: "#B33771"
        border.width: 3
    }

    onCheckedChanged: {
        console.log("Option 2 Status:", checked)
    }
}

// ---------- OPTION 3 ----------
```

```
        RadioButton {
            id: myRadioButton_3
            text: "Option 3"
            font.pointSize: 20
            font.bold: true
            height: 60
            ButtonGroup.group: radioGroup

            background: Rectangle {
                border.width: 3
                border.color: "#2C3A47"
                radius: 10
            }

            indicator: Rectangle {
                implicitWidth: 40
                implicitHeight: 40
                radius: width / 2
                x: 10
                y: (parent.height - height) / 2
                color: myRadioButton_3.checked ? "#EAB543" : "transparent"
                border.color: "#B33771"
                border.width: 3
            }

            onCheckedChanged: {
                console.log("Option 3 Status:", checked)
            }
        }
    }
}
```

OUTPUT:

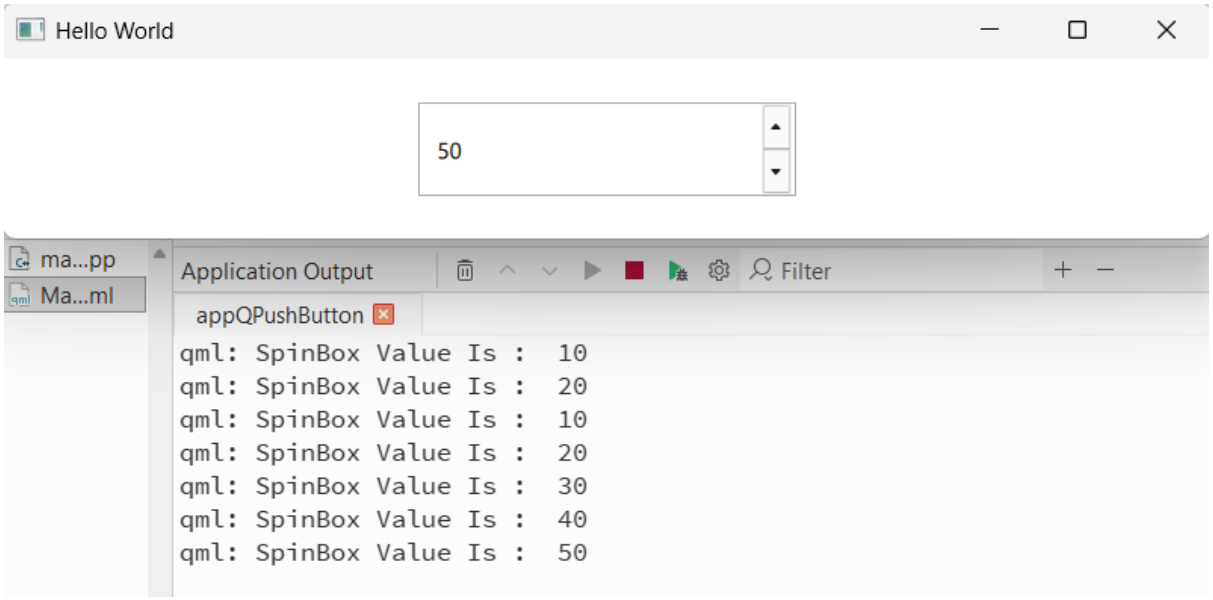


7. SpinBox Control:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    SpinBox
    {
        id: mySpinBox
        anchors.centerIn: parent
        from: 0 //minimum value
        to: 100 // maximum value
        stepSize: 10
```

```
        editable: true
        width: 200
        height: 50

        validator: IntValidator
        {
            bottom: mySpinBox.from
            top: mySpinBox.to
        }

        onValueChanged:
        {
            console.log("SpinBox Value Is : ", mySpinBox.value)
        }

    }

}
```

OUTPUT:

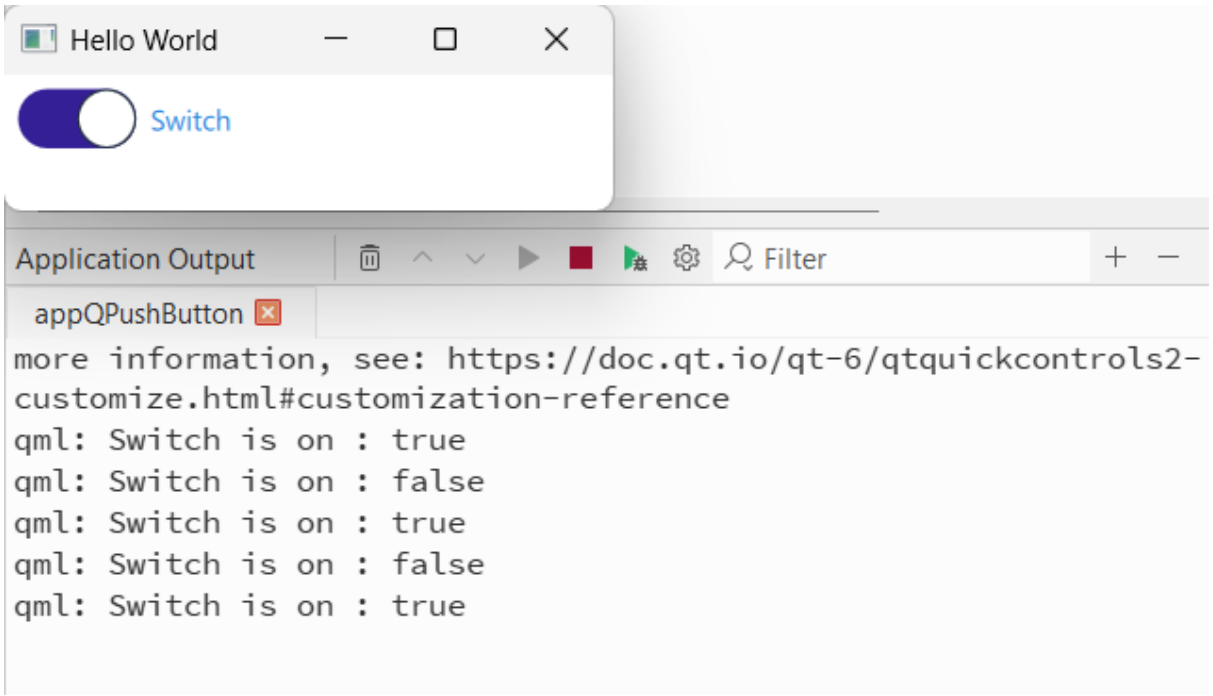


8. Switch Control:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Switch
    {
        id: mySwitch
        text: qsTr("Switch")

        indicator: Rectangle
        {
            implicitWidth: 50
            implicitHeight: 25
            x: mySwitch.leftPadding
            y: parent.height / 2 - height / 2
            radius: 15
            color: mySwitch.checked ? "#341f97" : "#ffffff"
            border.color: mySwitch.checked ? "#341f97" : "#222f3e"

            Rectangle
            {
                x: mySwitch.checked ? parent.width - width : 0
                width: 25
```

```
            height: 25
            radius: 15
            color: mySwitch.down ? "#222f3e" : "#ffffff"
            border.color: mySwitch.checked ? (mySwitch.down ? "#341f97" : "#222f3e") : "888888"
        }
    }

    contentItem: Text
    {
        text: mySwitch.text
        font: mySwitch.font
        color: mySwitch.down ? "#00d2d3" : "#2e86de"
        verticalAlignment: Text.AlignVCenter
        leftPadding: mySwitch.indicator.width + mySwitch.spacing

    }

    onCheckedChanged:
    {
        console.log("Switch is on :", checked)
    }
  }

}
```

OUTPUT:



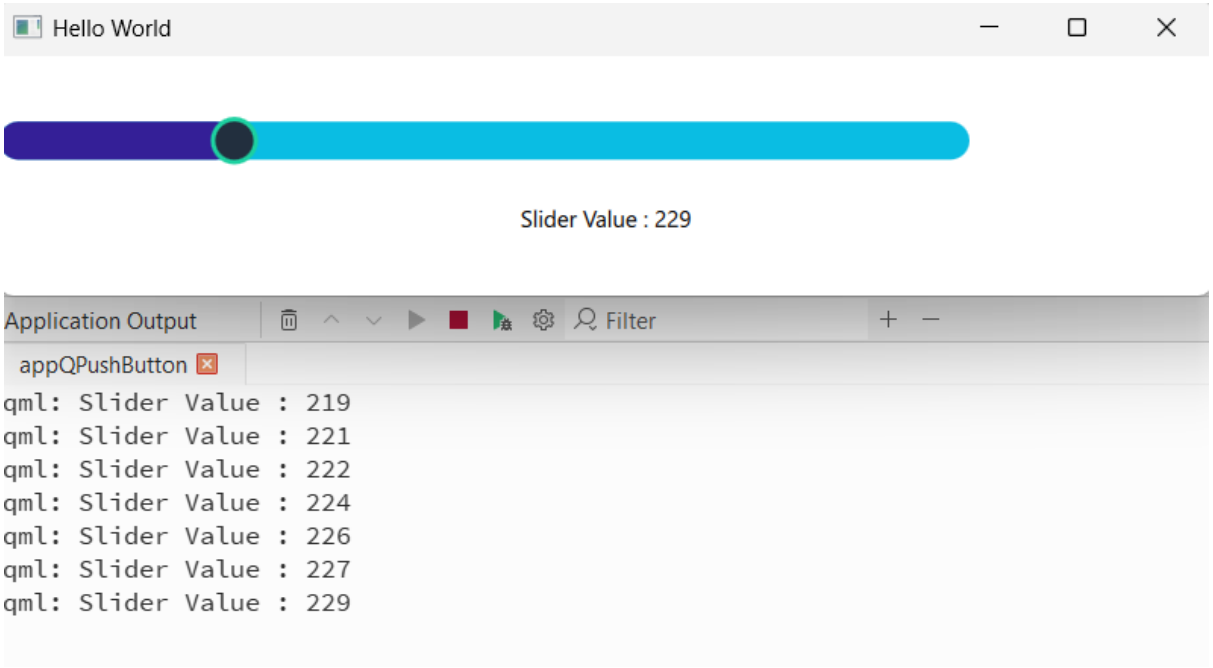
9. Slider Control:

```
import QtQuick 2.15

import QtQuick.Window 2.15

import QtQuick.Controls 2.15


Window {

   width: 640

   height: 480

   visible: true

   title: qsTr("Hello World")


   Column {

      spacing: 20

      anchors.centerIn: parent

      width: 640


      Slider {

         id: mySlider
```

```
width: parent.width * 0.8
from: 0
to: 1000
stepSize: 1

onValueChanged: {
    console.log("Slider Value :", value)
}

background: Rectangle {
    x: mySlider.leftPadding
    y: mySlider.topPadding + mySlider.availableHeight / 2 - height / 2
    implicitWidth: 200
    implicitHeight: 20
    width: mySlider.availableWidth
    height: implicitHeight
    radius: height / 2
    color: "#0abde3"

    Rectangle {
        width: mySlider.visualPosition === 0
                ? 0
                : mySlider.handle.x + mySlider.handle.width / 2
        height: parent.height
        color: "#341f97"
        radius: height / 2
    }
}

handle: Rectangle {
    x: mySlider.leftPadding + mySlider.visualPosition * (mySlider.availableWidth - width)
```

```
            y: mySlider.topPadding + mySlider.availableHeight / 2 - height / 2
            implicitHeight: 25
            implicitWidth: 25
            radius: implicitWidth / 2
            color: mySlider.pressed ? "#ee5253" : "#222f3e"
            border.color: "#1dd1a1"
            border.width: 2
        }
    }

    Text {
        id: mySlideText
        text: "Slider Value : " + mySlider.value
        anchors.horizontalCenter: parent.horizontalCenter
    }
  }
}
```

OUTPUT:

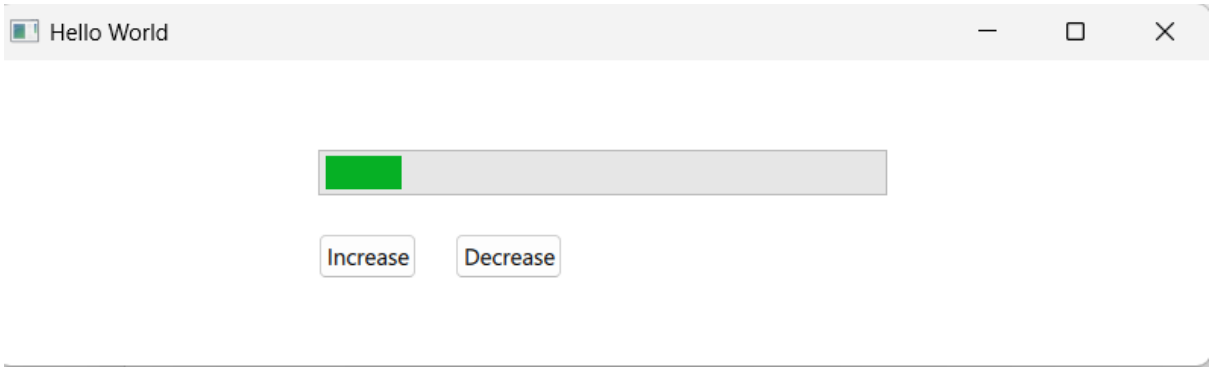


10. Progress Bar Control:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Column {
        spacing: 20
        anchors.centerIn: parent

        ProgressBar {
            id: myProgressBar
            width: 300
            value: 75
            from: 0      // Minimum Value
            to: 750      // Maximum Value
        }

        Row {
            spacing: 20

            Button {
                id: myButton_Inc
                text: "Increase"
```

```
            onClicked: {
                if (myProgressBar.value < myProgressBar.to) {
                    myProgressBar.value += 15
                }
            }
        }

        Button {
            id: myButton_Dec
            text: "Decrease"

            onClicked: {
                if (myProgressBar.value > myProgressBar.from) {
                    myProgressBar.value -= 15
                }
            }
        }
    }
  }
}
```

OUTPUT:

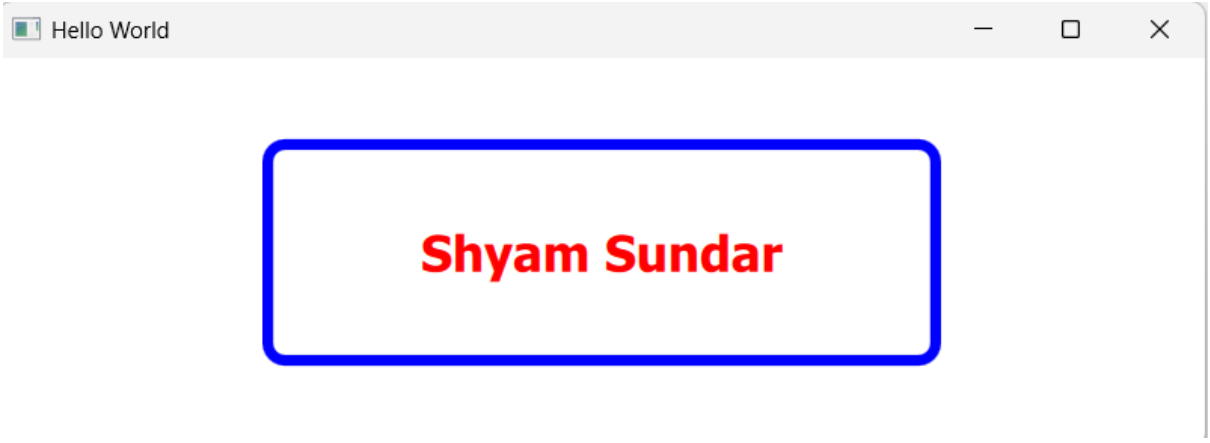


11. Label Control:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Label
    {
        id: myLabel
        font.pixelSize: 22
        font.bold: true
        font.italic: false
        font.family: "sundar"

        text: "Shyam Sundar"
        width: 300
        height: 100

        anchors.centerIn: parent

        verticalAlignment: Qt.AlignVCenter
        horizontalAlignment: Qt.AlignHCenter

        background: Rectangle
        {
```

```
        border.color: "blue"
        border.width: 5
        radius: 10
    }

    color: myMouseArea.containsMouse ? "red" : "black"
    scale: myMouseArea.containsMouse ? 1.2 : 1.0
    MouseArea
    {
        id: myMouseArea
        anchors.fill: parent
        hoverEnabled: true
        cursorShape: Qt.PointingHandCursor
    }
  }
}
```

OUTPUT:

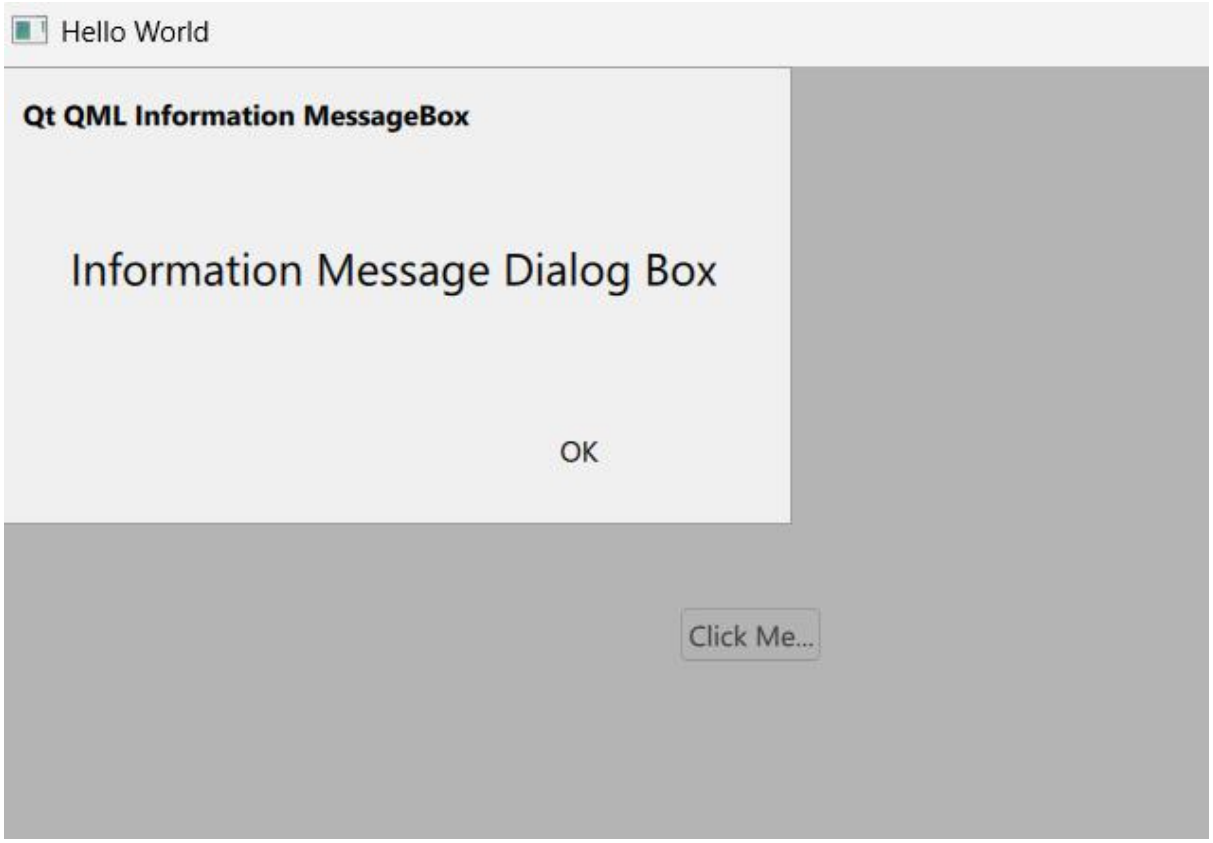


12. Message Dialog:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Button {
        anchors.centerIn: parent
        text: "Click Me..."

        onClicked: infoDialog.open()
    }

    Dialog {
        id: infoDialog
        title: "Qt QML Information MessageBox"
        modal: true
        standardButtons: Dialog.Ok

        contentItem: Text {
            text: "Information Message Dialog Box"
            padding: 20
            font.pointSize: 14
        }

        onAccepted: {
```

```
            console.log("Ok Button Clicked.")
        }
    }
}
```

OUTPUT:



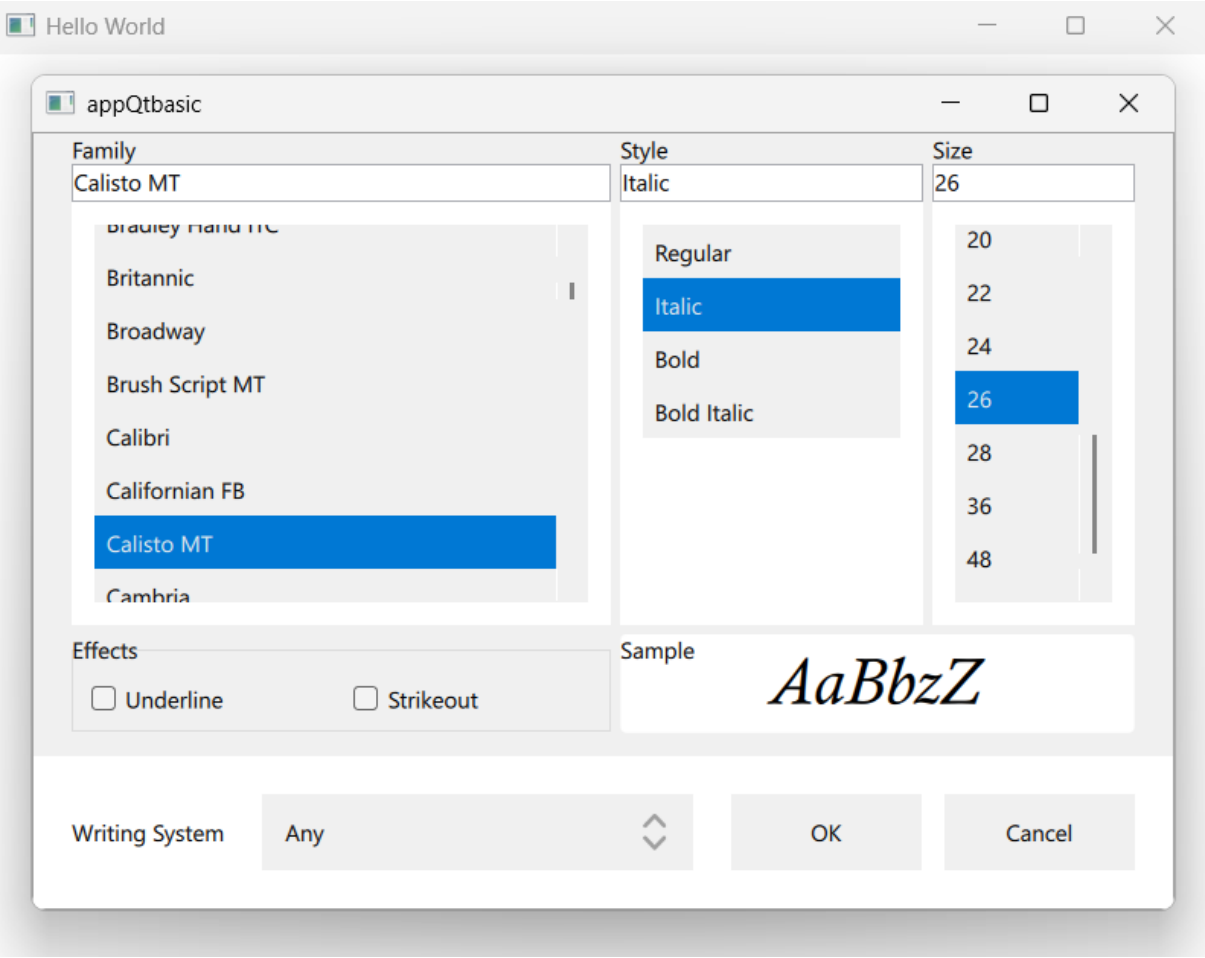
13. Font Dialog:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15
import QtQuick.Dialogs

Window {
    width: 640
    height: 480
    visible: true
```

```
title: qsTr("Hello World")

Button {
    id: myButton
    text: "Click Me"
    anchors.centerIn: parent
    font: selectedFont

    onClicked: {
        myFontDialog.open()
    }
}

// Store selected font here
property font selectedFont: Qt.font({
    family: "Arial",
    pointSize: 12
})

FontDialog {
    id: myFontDialog

    onAccepted: {
        selectedFont = selectedFontDialog.font

        console.log("Weight :", selectedFont.weight)
        console.log("Bold :", selectedFont.bold)
        console.log("Italic :", selectedFont.italic)
        console.log("Underline :", selectedFont.underline)
        console.log("Family :", selectedFont.family)
        console.log("Size :", selectedFont.pointSize)
```

```
        }
    }
}
```

OUTPUT:

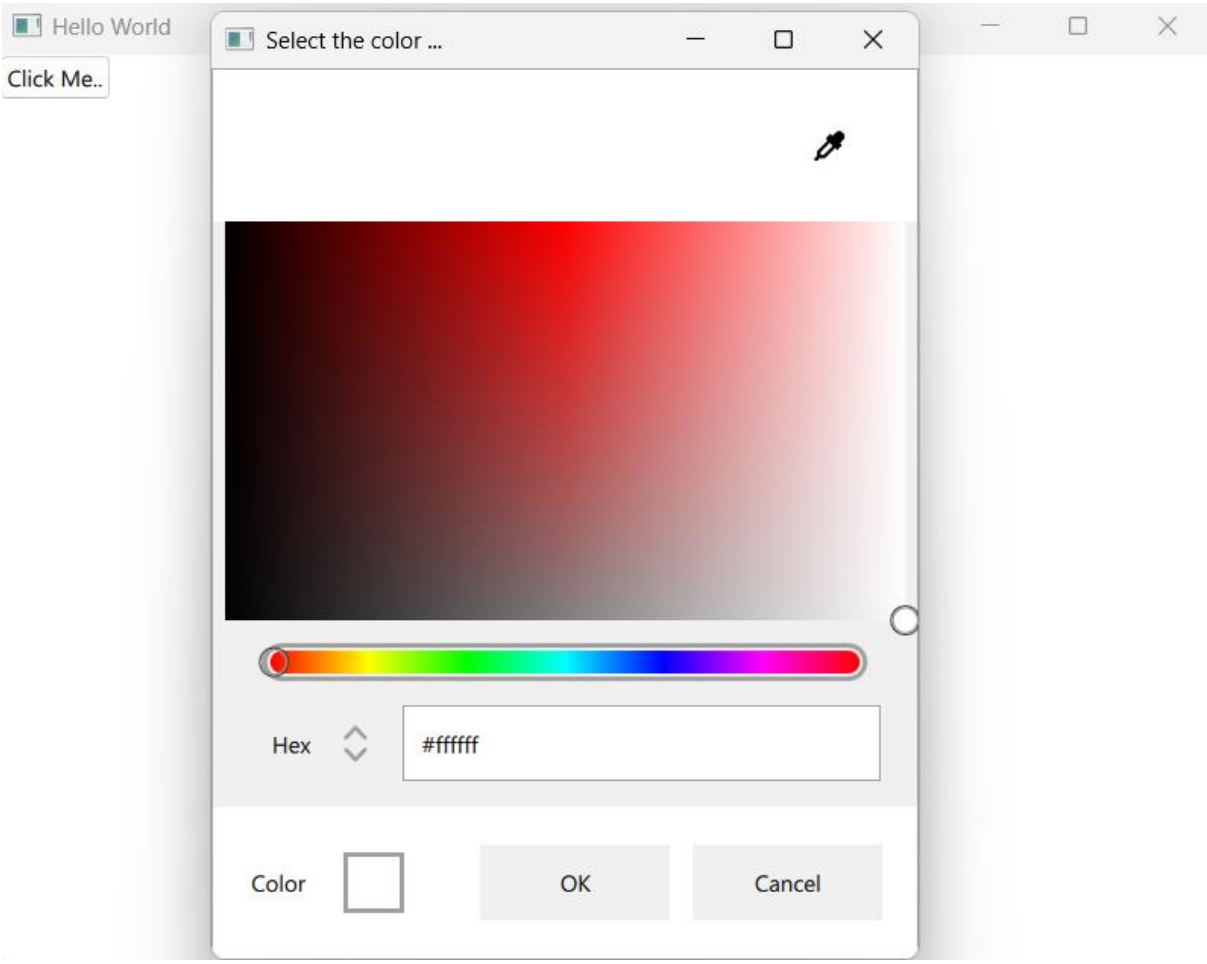


14. Color Dialog:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15
import QtQuick.Dialogs

Window {
    width: 640
    height: 480
```

```
    visible: true
    title: qsTr("Hello World")

    Column {
        Button {
            id: myButton
            text: "Click Me.."
            onClicked: {
                myColorDialog.open()
            }
        }

        Rectangle {
            id: myRect
            width: 200
            height: 200
        }
    }

    ColorDialog {
        id: myColorDialog
        title: "Select the color ..."
        onAccepted: {
            myRect.color = myColorDialog.color
        }
    }
}
```

OUTPUT:

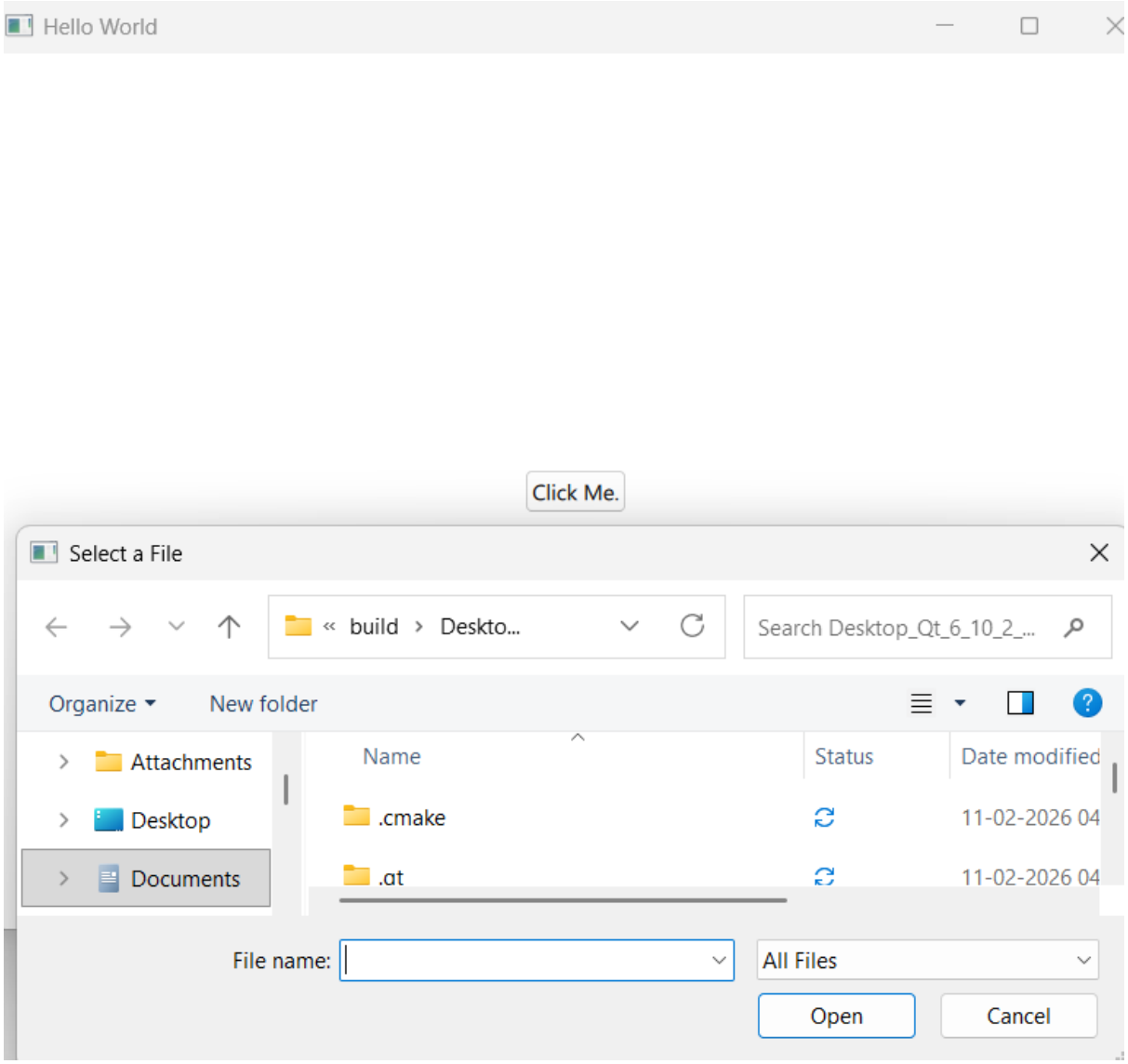


15. File Dialog:

```
import QtQuick 2.15
import QtQuick.Window 2.15
import QtQuick.Controls 2.15
import QtQuick.Dialogs 6.3    // <-- Important: use Dialogs 6.x

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Button {
```

```
        id: myButton
        text: "Click Me."
        anchors.centerIn: parent
        onClicked: {
            myFileDialog.open()
        }
    }

    FileDialog {
        id: myFileDialog
        title: "Select a File"
        currentFolder: StandardPaths.pictures  // replaces shortcuts.pictures

        onAccepted: {
            console.log("Selected file:", selectedFile)
        }

        onRejected: {
            console.log("Dialog canceled")
        }
    }
}
```

OUTPUT:

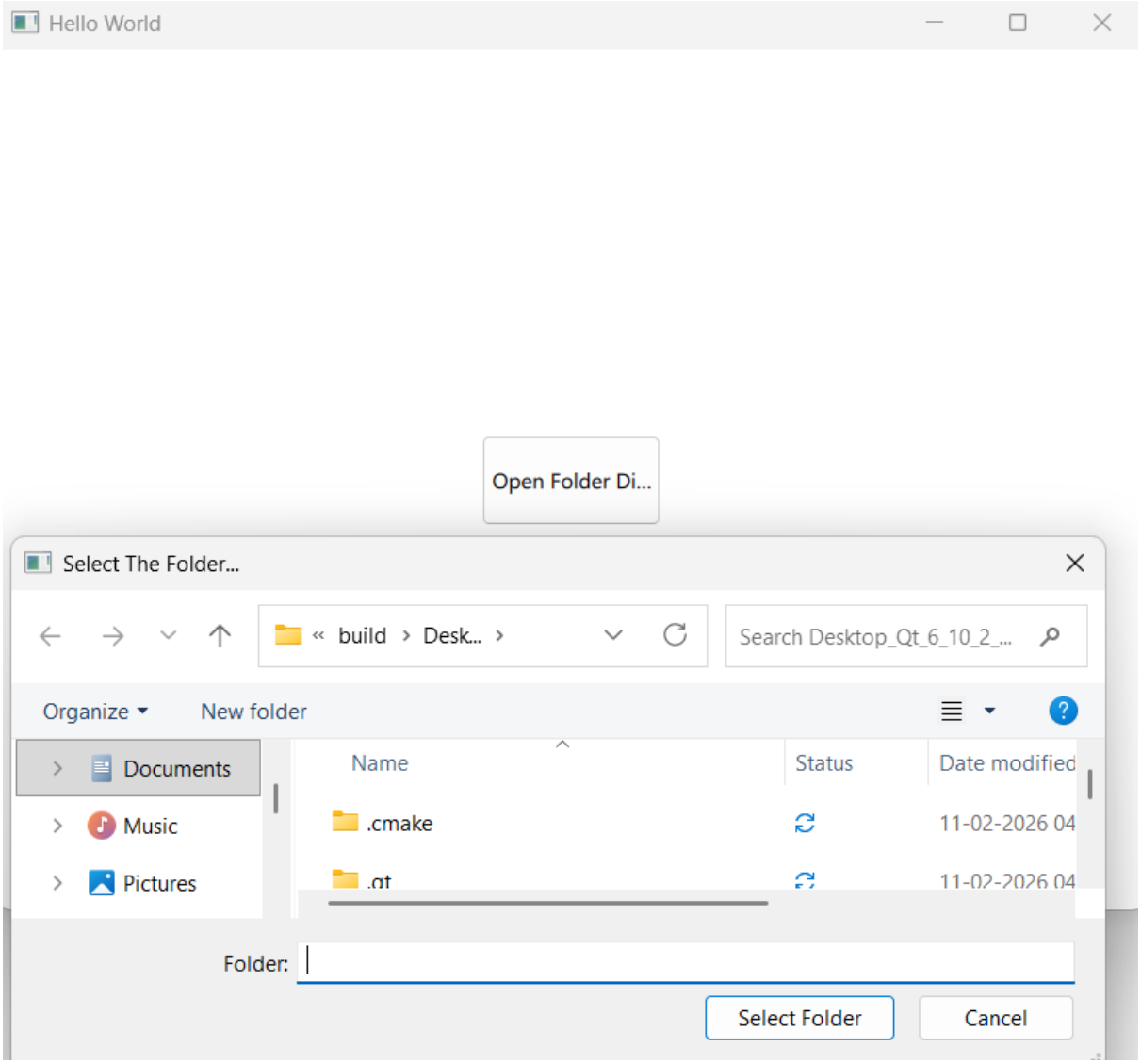


16. Folder Dialog:

```
import QtQuick
import QtQuick.Controls
import QtQuick.Dialogs

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")
```

```
    Button {
        id: myButton
        width: 100
        height: 50
        text: "Open Folder Dialog Control"
        anchors.centerIn: parent
        onClicked: {
            myFolderDialog.open()
        }
    }

    FolderDialog {
        id: myFolderDialog
        title: "Select The Folder..."
        onAccepted: {
            console.log(myFolderDialog.selectedFolder)
        }
    }
}
```

OUTPUT:



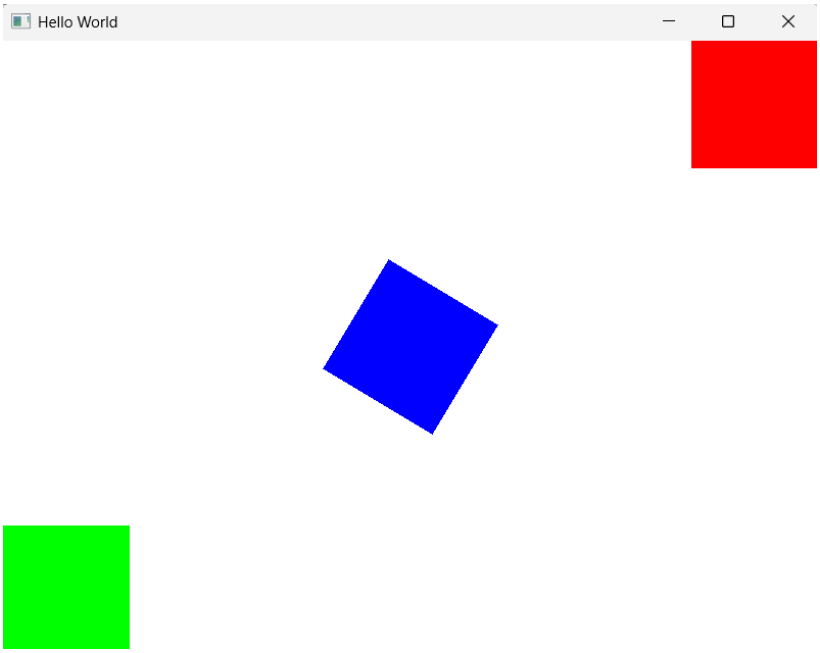
17. Property Animation:

```
import QtQuick

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Rectangle
```

```
{
    id:myRect_1
    width: 100
    height: 100
    color: "#FF0000"

    PropertyAnimation on x
    {
        from: 0
        to: width - myRect_1.width
        duration: 1500
        easing.type: Easing.InBounce
    }
}

Rectangle
{
    id:myRect_2
    width: 100
    height: 100
    color: "#00FF00"

    PropertyAnimation on y
    {
        from: 0
        to: height - myRect_2.height
        duration: 2000
        easing.type: Easing.OutBounce
    }
}
```

```
    Rectangle
    {
        id:myRect_3
        width: 100
        height: 100
        color: "#0000FF"

        anchors.centerIn: parent

        PropertyAnimation on rotation
        {
            from: 0
            to: 360
            loops: Animation.Infinite
            duration: 2000
            easing.type: Easing.InOutBounce
        }
    }
}
```

OUTPUT:



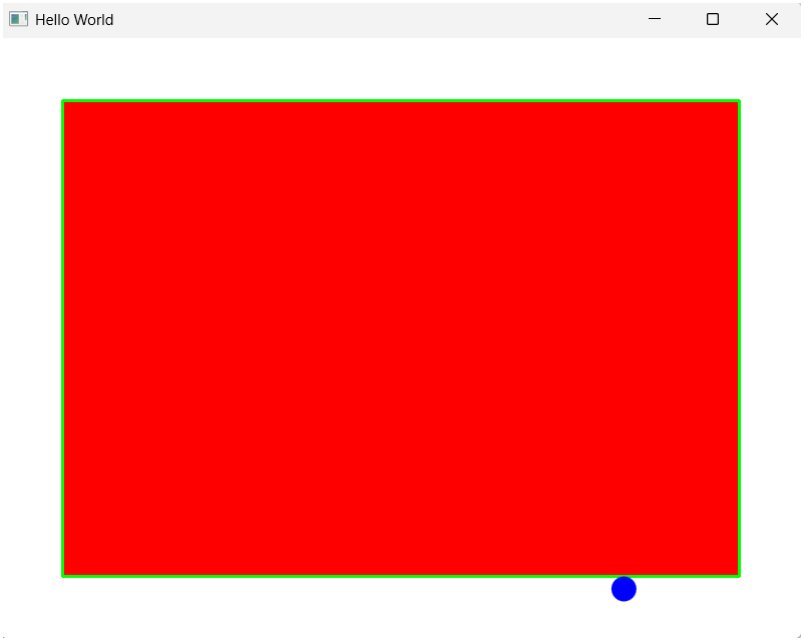
18. Path Animation:

```
import QtQuick
import QtQuick.Window
import QtQuick.Shapes

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Shape {
        id: myShape
        width: parent.width
        height: parent.height

        ShapePath {
            strokeColor: "#00FF00"
            strokeWidth: 2
```

```
        fillColor: "#FF0000"   // fixed

        startX: 50
        startY: 50

        PathLine { x: 590; y: 50 }
        PathLine { x: 590; y: 430 }
        PathLine { x: 50; y: 430 }
        PathLine { x: 50; y: 50 }
    }
}

Rectangle
{
    id:circle
    width: 20
    height: 20
    radius: circle.height / 2
    color: "#0000FF"
    x: 50; y:50
}

Path
{
    id: myPath
    startX: 50; startY: 50

    PathLine { x: 590; y: 50 }
    PathLine { x: 590; y: 430 }
    PathLine { x: 50; y: 430 }
    PathLine { x: 50; y: 50 }
```

```
    }

    PathAnimation
    {
        id:myPathAnimation
        target: circle
        path: myPath
        duration: 2000
        loops: Animation.Infinite
        running: true

    }
}
```

OUTPUT:



19. Sequential Animation:

```
import QtQuick
import QtQuick.Window
import QtQuick.Shapes
```

```
Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Rectangle
    {
        id:myRect
        width: 200
        height: 200
        color: "#FF0000"
    }

    SequentialAnimation
    {
        id:mySeqAnimation
        running: true
        loops: Animation.Infinite

        NumberAnimation
        {
            target: myRect
            property: "width"
            from: 200
            to: 300
            duration: 2000
            easing.type: Easing.InOutQuad
        }

        NumberAnimation
```

```
        {
            target: myRect
            property: "height"
            from: 200
            to: 300
            duration: 2000
            easing.type: Easing.InOutQuad
        }

        NumberAnimation
        {
            target: myRect
            property: "width"
            from: 200
            to: 300
            duration: 2000
            easing.type: Easing.InOutQuad
        }
    }
}
```

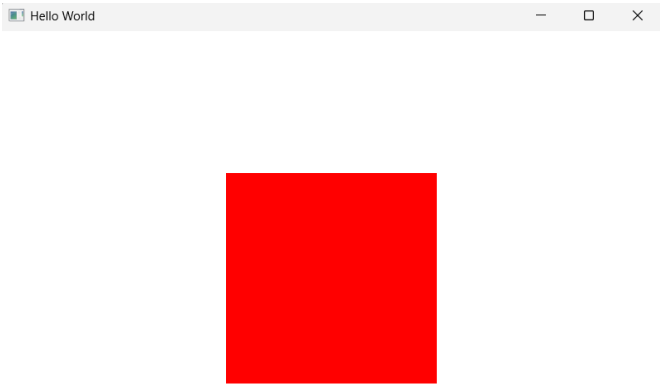
OUTPUT:

20. Parallel Animation:

```
import QtQuick
import QtQuick.Window
import QtQuick.Shapes

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Rectangle
    {
        id:myRect
        width: 200
        height: 200
        color: "#FF0000"
        anchors.centerIn: parent
```

```
    }

    ParallelAnimation
    {
        id:myPara_1
        running: true
        loops: Animation.Infinite

        NumberAnimation
        {
            target: myRect
            property: "width"
            from: 200
            to: 300
            duration: 2000
        }

        NumberAnimation
        {
            target: myRect
            property: "height"
            from: 200
            to: 300
            duration: 2000
        }
    }
}
```

OUTPUT:

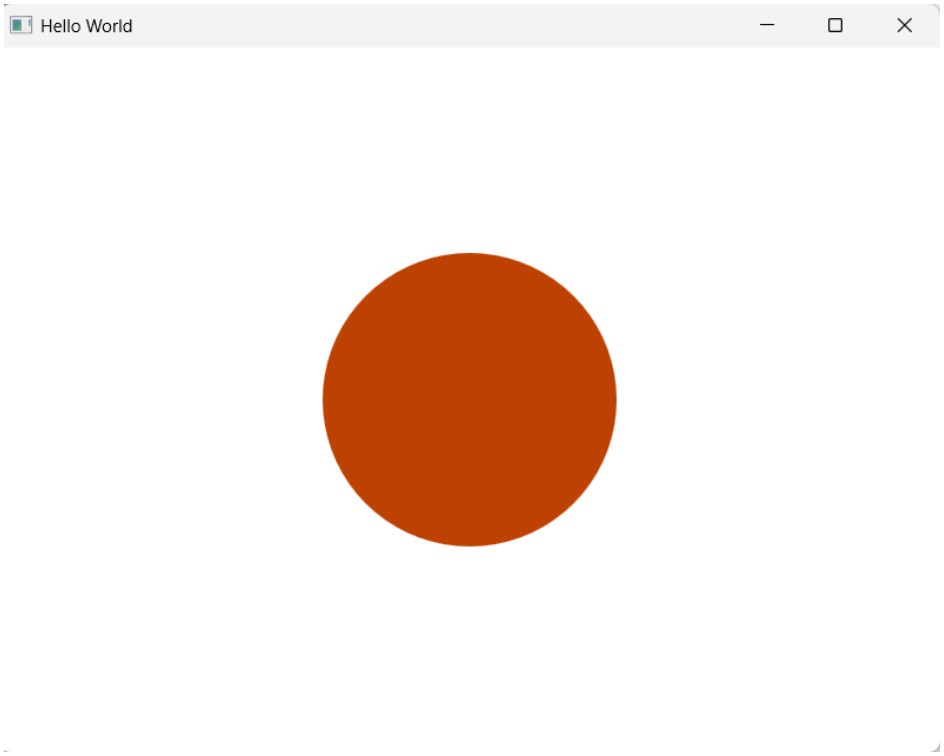


21. Color Animation:

```
import QtQuick
import QtQuick.Window
import QtQuick.Shapes

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Rectangle
    {
        id:myRect
        width: 200
        height: 200
        anchors.centerIn: parent
        color: "#FF0000"
        radius: height / 2
    }
```

```
    ColorAnimation
    {
        target: myRect
        property: "color"
        from: "#FF0000"
        to: "#00FF00"
        duration: 2000
        loops: Animation.Infinite
        running: true
    }
}
```

OUTPUT:



22. Column Layout:

```
import QtQuick
import QtQuick.Window
import QtQuick.Layouts

Window {
```

```
width: 640
height: 480
visible: true
title: qsTr("Hello World")

ColumnLayout
{
    anchors.centerIn: parent
    spacing: 10

    Rectangle
    {
        id: myRect_1
        Layout.preferredWidth: 100
        Layout.preferredHeight: 100
        color: "#EE5A24"
        Layout.alignment: Qt.AlignCenter
        Layout.fillWidth: true
        Text {
            id: myRect_Name_1
            text: qsTr("Rect 1")
        }
    }

    Rectangle
    {
        id: myRect_2
        Layout.preferredWidth: 100
        Layout.preferredHeight: 100
        color: "#009432"
        Layout.alignment: Qt.AlignRight
```

```
    Layout.fillWidth: true
    Text {
        id: myRect_Name_2
        text: qsTr("Rect 2")
    }
}

Rectangle
{
    id: myRect_3
    Layout.preferredWidth: 100
    Layout.preferredHeight: 100
    color: "#A3CB38"
    Layout.alignment: Qt.AlignLeft
    Layout.fillWidth: true
    Text {
        id: myRect_Name_3
        text: qsTr("Rect 3")
    }
}

Rectangle
{
    id: myRect_4
    Layout.preferredWidth: 100
    Layout.preferredHeight: 100
    color: "#1289A7"
    Layout.alignment: Qt.AlignCenter
    Layout.fillWidth: true
    Text {
        id: myRect_Name_4
```

```
            text: qsTr("Rect 4")
        }
    }
  }
}
```

OUTPUT:

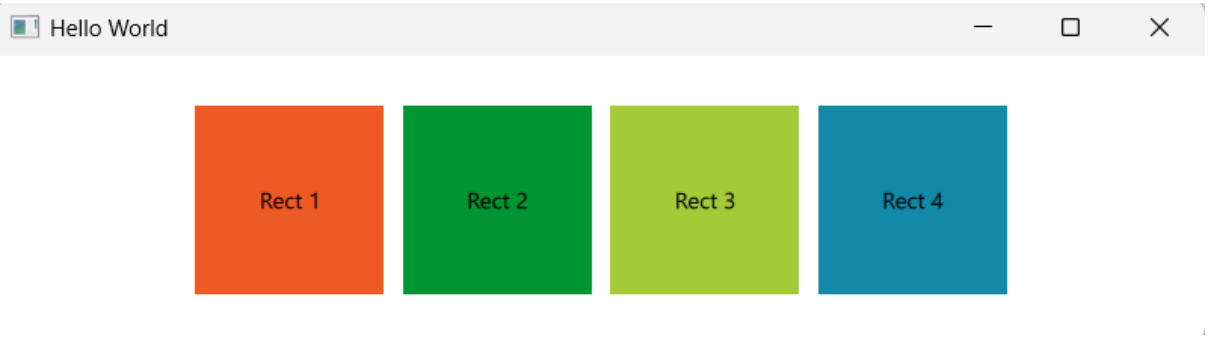
23. Row Layout:

```
import QtQuick
import QtQuick.Window
import QtQuick.Layouts

Window {
  width: 640
  height: 480
  visible: true
  title: qsTr("Hello World")

  RowLayout {
```

```
anchors.centerIn: parent
spacing: 10

Rectangle {
    id: myRect_1
    Layout.preferredWidth: 100
    Layout.preferredHeight: 100
    color: "#EE5A24"
    Layout.alignment: Qt.AlignVCenter
    Text {
        id: myRect_Name_1
        anchors.centerIn: parent
        text: qsTr("Rect 1")
    }
}

Rectangle {
    id: myRect_2
    Layout.preferredWidth: 100
    Layout.preferredHeight: 100
    color: "#009432"
    Layout.alignment: Qt.AlignVCenter
    Text {
        id: myRect_Name_2
        anchors.centerIn: parent
        text: qsTr("Rect 2")
    }
}

Rectangle {
    id: myRect_3
```

```
            Layout.preferredWidth: 100
            Layout.preferredHeight: 100
            color: "#A3CB38"
            Layout.alignment: Qt.AlignVCenter
            Text {
                id: myRect_Name_3
                anchors.centerIn: parent
                text: qsTr("Rect 3")
            }
        }

        Rectangle {
            id: myRect_4
            Layout.preferredWidth: 100
            Layout.preferredHeight: 100
            color: "#1289A7"
            Layout.alignment: Qt.AlignVCenter
            Text {
                id: myRect_Name_4
                anchors.centerIn: parent
                text: qsTr("Rect 4")
            }
        }
    }
}
```

OUTPUT:



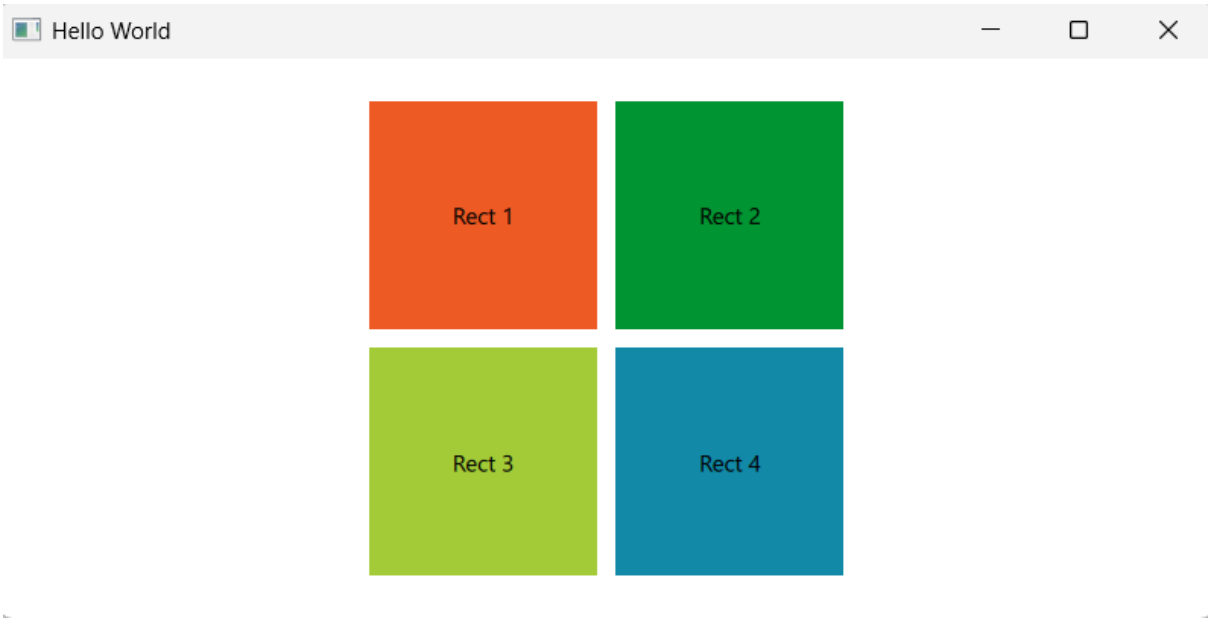
24. Grid Layout:

```
import QtQuick
import QtQuick.Window
import QtQuick.Layouts

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    GridLayout {
        anchors.centerIn: parent
        rows: 2
        columns: 2
        rowSpacing: 10
        columnSpacing: 10

        Rectangle {
            id: myRect_1
            Layout.preferredWidth: 120
            Layout.preferredHeight: 120
            color: "#EE5A24"
            Text {
```

```
        anchors.centerIn: parent
        text: "Rect 1"
    }
}

Rectangle {
    id: myRect_2
    Layout.preferredWidth: 120
    Layout.preferredHeight: 120
    color: "#009432"
    Text {
        anchors.centerIn: parent
        text: "Rect 2"
    }
}

Rectangle {
    id: myRect_3
    Layout.preferredWidth: 120
    Layout.preferredHeight: 120
    color: "#A3CB38"
    Text {
        anchors.centerIn: parent
        text: "Rect 3"
    }
}

Rectangle {
    id: myRect_4
    Layout.preferredWidth: 120
    Layout.preferredHeight: 120
```

```
        color: "#1289A7"
        Text {
            anchors.centerIn: parent
            text: "Rect 4"
        }
    }
  }
}
```

OUTPUT:



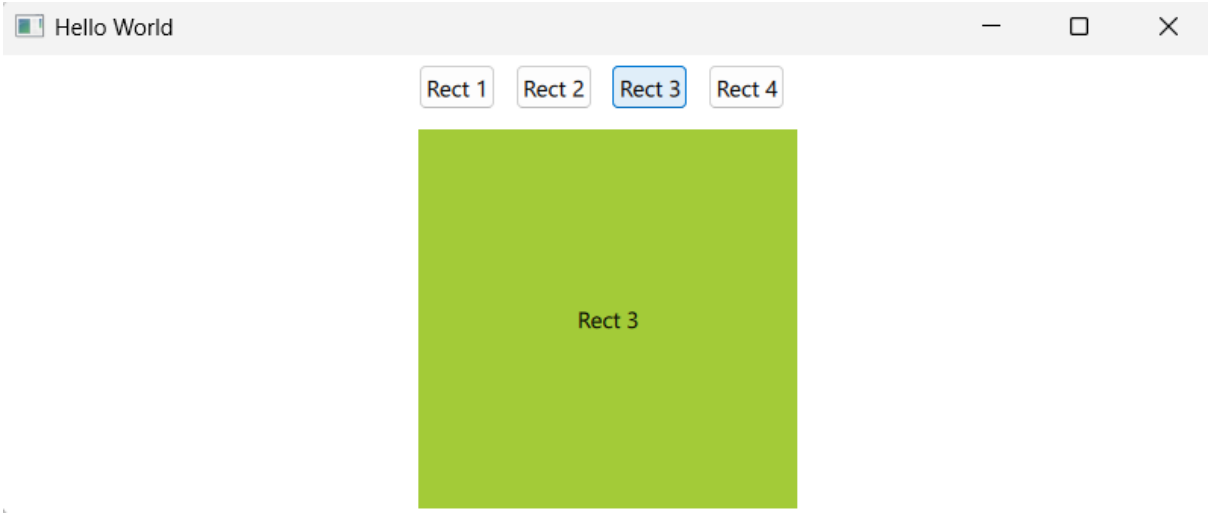
25. Stack Layout:

```
import QtQuick
import QtQuick.Window
import QtQuick.Layouts
import QtQuick.Controls

Window {
  width: 640
  height: 480
  visible: true
```

```
title: qsTr("Hello World")

Column {
    anchors.centerIn: parent
    spacing: 10

    // --- Buttons to switch pages ---
    Row {
        spacing: 10
        Button { text: "Rect 1"; onClicked: stack.currentIndex = 0 }
        Button { text: "Rect 2"; onClicked: stack.currentIndex = 1 }
        Button { text: "Rect 3"; onClicked: stack.currentIndex = 2 }
        Button { text: "Rect 4"; onClicked: stack.currentIndex = 3 }
    }

    // ------------ STACK LAYOUT ------------
    StackLayout {
        id: stack
        width: 200
        height: 200

        Rectangle {
            color: "#EE5A24"
            Text { anchors.centerIn: parent; text: "Rect 1" }
        }

        Rectangle {
            color: "#009432"
            Text { anchors.centerIn: parent; text: "Rect 2" }
        }
```

```
        Rectangle {
            color: "#A3CB38"
            Text { anchors.centerIn: parent; text: "Rect 3" }
        }

        Rectangle {
            color: "#1289A7"
            Text { anchors.centerIn: parent; text: "Rect 4" }
        }
    }
  }
}
```

OUTPUT:



26. Stack Layout:

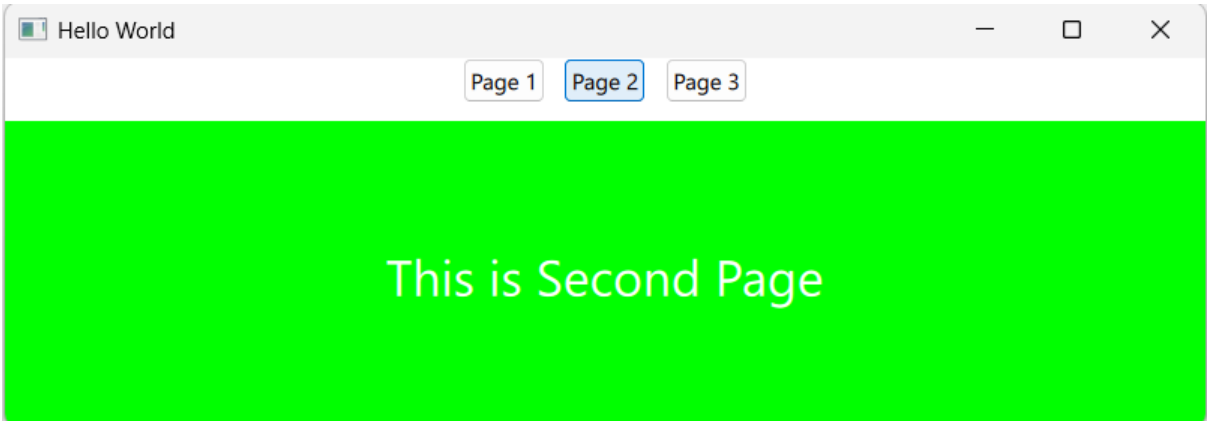
```
import QtQuick
import QtQuick.Window
import QtQuick.Layouts
import QtQuick.Controls

Window {
  width: 640
  height: 480
```

```
visible: true
title: qsTr("Hello World")

ColumnLayout {
    anchors.fill: parent
    spacing: 10

    // ---------------- BUTTONS ----------------
    RowLayout {
        Layout.alignment: Qt.AlignHCenter
        spacing: 10

        Button {
            text: "Page 1"
            onClicked: stack.currentIndex = 0
        }
        Button {
            text: "Page 2"
            onClicked: stack.currentIndex = 1
        }
        Button {
            text: "Page 3"
            onClicked: stack.currentIndex = 2
        }
    }

    // ---------------- STACK LAYOUT ----------------
    StackLayout {
        id: stack
        Layout.fillHeight: true
        Layout.fillWidth: true
```

```
currentIndex: 0

Rectangle {
    anchors.fill: parent
    color: "#FF0000"
    Label {
        anchors.centerIn: parent
        text: "This is First Page"
        font.pointSize: 20
        color: "#FFFFFF"
    }
}

Rectangle {
    anchors.fill: parent
    color: "#00FF00"
    Label {
        anchors.centerIn: parent
        text: "This is Second Page"
        font.pointSize: 20
        color: "#FFFFFF"
    }
}

Rectangle {
    anchors.fill: parent
    color: "#0000FF"
    Label {
        anchors.centerIn: parent
        text: "This is Third Page"
        font.pointSize: 20
```

```
                color: "#FFFFFF"
            }
        }
    }
  }
}
```

OUTPUT:



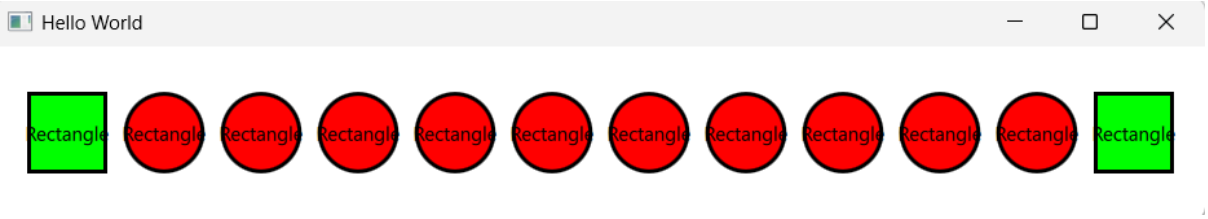
27. Repeater:

```
import QtQuick
import QtQuick.Window
import QtQuick.Layouts
import QtQuick.Controls

Window {
  width: 640
  height: 480
  visible: true
  title: qsTr("Hello World")

  RowLayout {
    anchors.centerIn: parent
    spacing: 10
```

```
Rectangle {
    width: 50
    height: 50
    color: "#00FF00"
    border.width: 2
    Text {
        text: "Rectangle"
        anchors.centerIn: parent
    }
}


Repeater {
    model: 10
    Rectangle {
        width: 50
        height: 50
        color: "#FF0000"
        border.width: 2
        radius: 25   // circle shape
        Text {
            text: "Rectangle"
            anchors.centerIn: parent
        }
    }
}
Rectangle {
    width: 50
    height: 50
```

```
        color: "#00FF00"
        border.width: 2
        Text {
            text: "Rectangle"
            anchors.centerIn: parent
        }
    }
  }
}
```

OUTPUT:

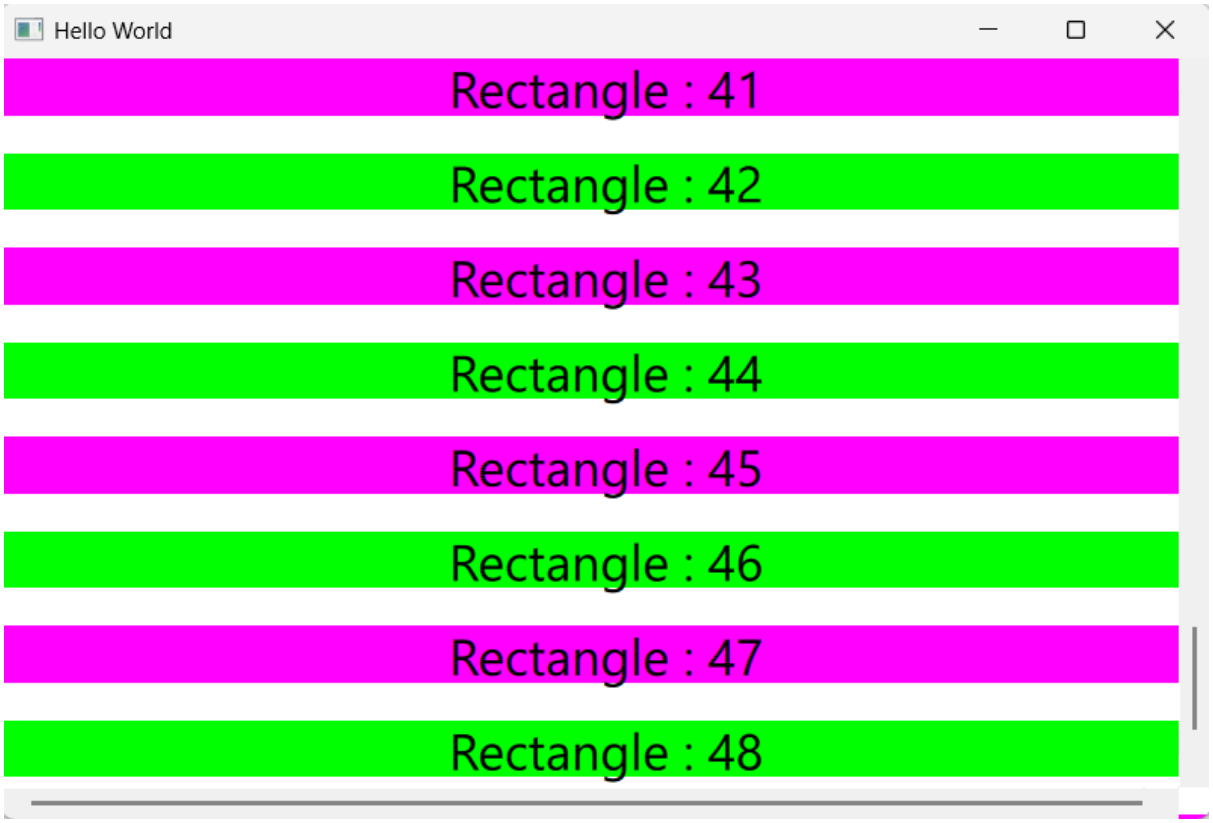


28. Scroll View Control:

```
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts

Window {
  width: 640
  height: 480
  visible: true
  title: qsTr("Hello World")

  ScrollView {
    anchors.fill: parent

    ScrollBar.horizontal.policy : ScrollBar.AlwaysOn
```

```
        ScrollBar.vertical.policy : ScrollBar.AlwaysOn



        ColumnLayout {
            spacing: 20
            width: parent.width

            Repeater {
                model: 50

                Rectangle {
                    width: parent.width
                    height: 30
                    color: index % 2 == 0 ? "#FF00FF" : "#00FF00"

                    Text {
                        id: myText
                        anchors.centerIn: parent
                        text: "Rectangle : " + (index + 1)
                        font.pointSize: 20
                    }
                }
            }
        }
    }
}
```

OUTPUT:



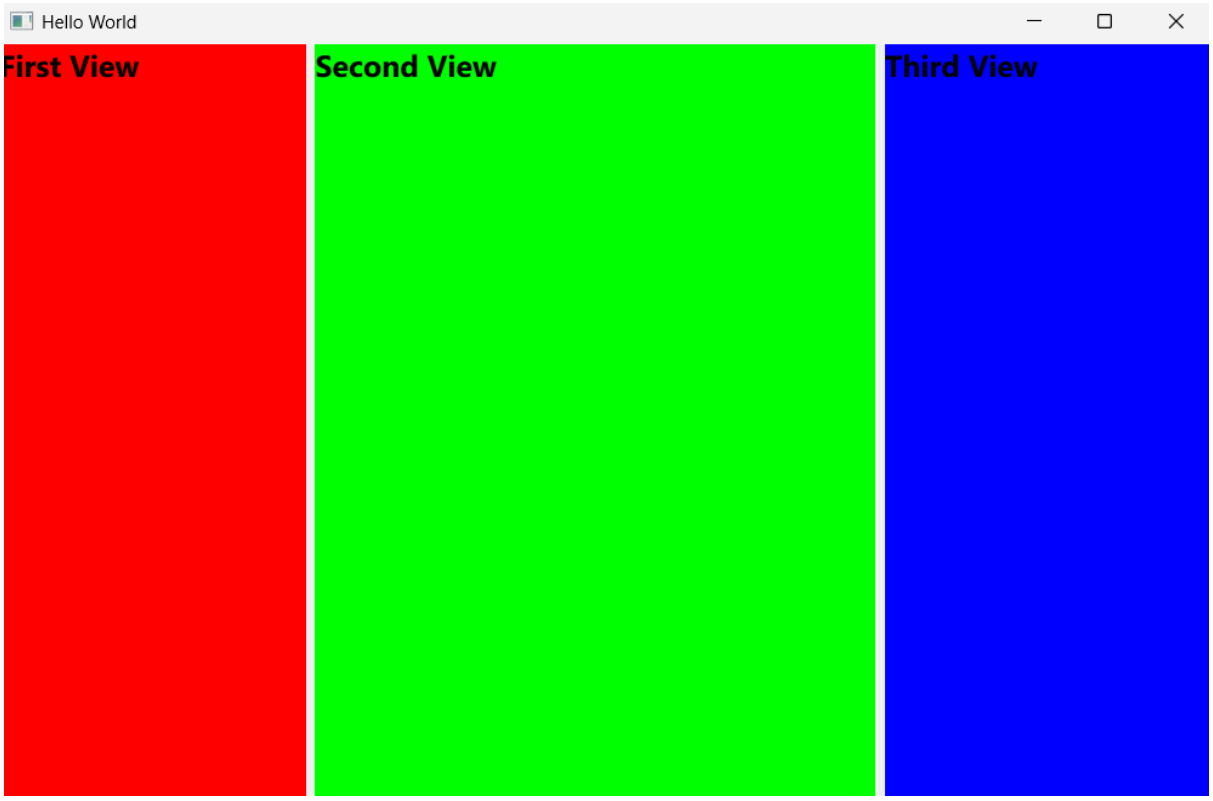
29. Split View Control:

```
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts

Window {
    width: 800
    height: 500
    visible: true
    title: qsTr("Hello World")

    SplitView
    {
        id:mySplitView
        anchors.fill: parent
```

```
Rectangle
{
    width: 200
    color: "#FF0000"
    Text {
        id: myRectText_1
        text: "First View"
        font.bold: true
        font.pointSize: 15
    }
}

Rectangle
{
    width: 200
    color: "#00FF00"
    Text {
        id: myRectText_2
        text: "Second View"
        font.bold: true
        font.pointSize: 15
    }
}

Rectangle
{
    width: 200
    color: "#0000FF"
    Text {
        id: myRectText_3
```

```
            text: "Third View"
            font.bold: true
            font.pointSize: 15
        }
    }
  }
}
```

OUTPUT:



30. StackView Control:

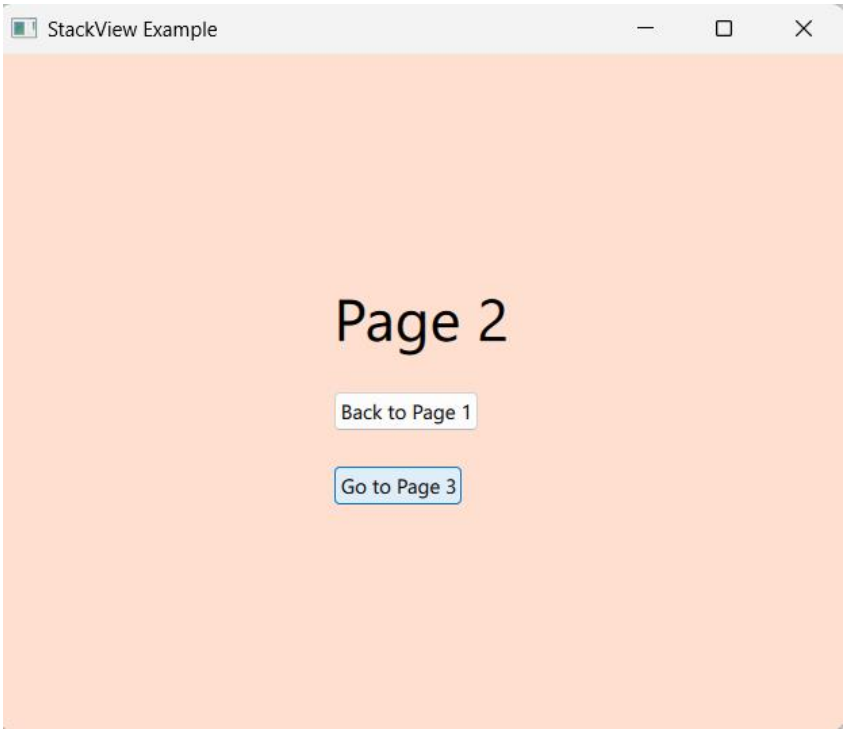
```
import QtQuick
import QtQuick.Controls

ApplicationWindow {
  width: 500
  height: 400
```

```
visible: true
title: "StackView Example"

StackView {
    id: stackView
    anchors.fill: parent

    initialItem: page1
}

// ---------- PAGE 1 ----------
Component {
    id: page1
    Rectangle {
        color: "#D0F0FF"
        anchors.fill: parent

        Column {
            anchors.centerIn: parent
            spacing: 20

            Text {
                text: "Page 1"
                font.pointSize: 26
            }

            Button {
                text: "Go to Page 2"
                onClicked: stackView.push(page2)
            }
        }
```

```
    }
}

// ---------- PAGE 2 ----------
Component {
    id: page2
    Rectangle {
        color: "#FFE0D0"
        anchors.fill: parent

        Column {
            anchors.centerIn: parent
            spacing: 20

            Text {
                text: "Page 2"
                font.pointSize: 26
            }

            Button {
                text: "Back to Page 1"
                onClicked: stackView.pop()
            }

            Button {
                text: "Go to Page 3"
                onClicked: stackView.push(page3)
            }
        }
    }
}
```

```
    // ---------- PAGE 3 ----------
    Component {
        id: page3
        Rectangle {
            color: "#D0FFD0"
            anchors.fill: parent

            Column {
                anchors.centerIn: parent
                spacing: 20

                Text {
                    text: "Page 3"
                    font.pointSize: 26
                }

                Button {
                    text: "Back"
                    onClicked: stackView.pop()
                }
            }
        }
    }
}
```

OUTPUT:



31. Swipe View Control:

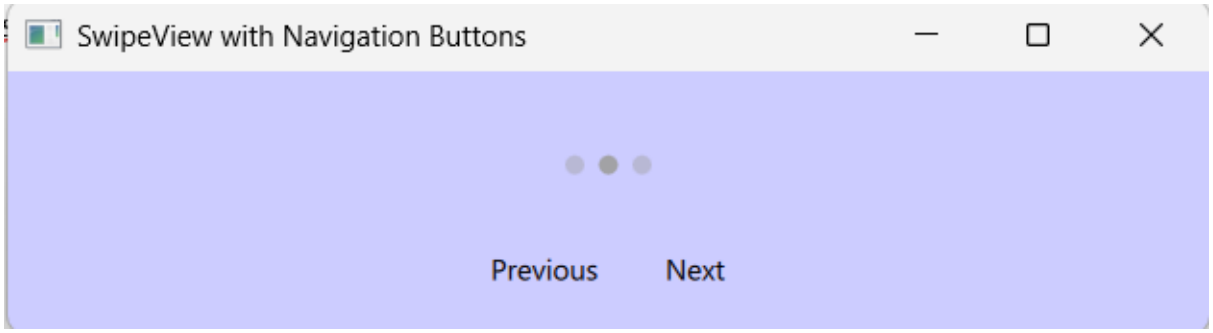
```
import QtQuick
import QtQuick.Controls

ApplicationWindow {
    width: 500
    height: 300
    visible: true
    title: "SwipeView with Navigation Buttons"

    SwipeView {
        id: swipeView
        anchors.fill: parent
        currentIndex: 0

        // PAGE 1
```

```
Rectangle {
    color: "#FFCCCC"
    anchors.fill: parent
    Text {
        text: "Page 1"
        anchors.centerIn: parent
        font.pointSize: 24
    }
}

// PAGE 2
Rectangle {
    color: "#CCFFCC"
    anchors.fill: parent
    Text {
        text: "Page 2"
        anchors.centerIn: parent
        font.pointSize: 24
    }
}

// PAGE 3
Rectangle {
    color: "#CCCCFF"
    anchors.fill: parent
    Text {
        text: "Page 3"
        anchors.centerIn: parent
        font.pointSize: 24
    }
}
```

```
}

// --- BOTTOM NAVIGATION BUTTONS ---
Row {
    spacing: 20
    anchors.horizontalCenter: parent.horizontalCenter
    anchors.bottom: parent.bottom
    anchors.bottomMargin: 15

    Button {
        text: "Previous"
        enabled: swipeView.currentIndex > 0

        onClicked: {
            if (swipeView.currentIndex > 0)
                swipeView.currentIndex--
        }
    }

    Button {
        text: "Next"
        enabled: swipeView.currentIndex < swipeView.count - 1

        onClicked: {
            if (swipeView.currentIndex < swipeView.count - 1)
                swipeView.currentIndex++
        }
    }
}

// --- PAGE INDICATOR ---
```

```
    PageIndicator {
        count: swipeView.count
        currentIndex: swipeView.currentIndex

        anchors.horizontalCenter: parent.horizontalCenter
        anchors.bottom: parent.bottom
        anchors.bottomMargin: 60
    }
}
```

OUTPUT:



32. Resource Collection File (.qrc):

<u>Main.qml:</u>

```
import QtQuick
import QtQuick.Controls

Window {
    width: 500
    height: 300
    visible: true
    title: "Hello World"

    Image {
        id: myImage
```

```
        source: "qrc:/Screenshot 2025-09-27 191307.jpg"
        width: parent.width
        height: parent.height
    }
}
```

CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.16)

project(untitled VERSION 0.1 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD_REQUIRED ON)

find_package(Qt6 REQUIRED COMPONENTS Quick)

qt_standard_project_setup(REQUIRES 6.8)

qt_add_executable(appuntitled
    main.cpp
)

qt_add_qml_module(appuntitled
    URI untitled
    VERSION 1.0
    QML_FILES
        Main.qml
        RESOURCES Resource.qrc
)

qt_add_resources(appQt_Resource "icons"
    PREFIX "/"
    FILE Image/Screenshot 2025-09-27 191307.png
```

```
)
# Qt for iOS sets MACOSX_BUNDLE_GUI_IDENTIFIER automatically since Qt 6.1.
# If you are developing for iOS or macOS you should consider setting an
# explicit, fixed bundle identifier manually though.
set_target_properties(appuntitled PROPERTIES
#    MACOSX_BUNDLE_GUI_IDENTIFIER com.example.appuntitled
    MACOSX_BUNDLE_BUNDLE_VERSION ${PROJECT_VERSION}
    MACOSX_BUNDLE_SHORT_VERSION_STRING
${PROJECT_VERSION_MAJOR}.${PROJECT_VERSION_MINOR}
    MACOSX_BUNDLE TRUE
    WIN32_EXECUTABLE TRUE
)

target_link_libraries(appuntitled
    PRIVATE Qt6::Quick
)

include(GNUInstallDirs)
install(TARGETS appuntitled
    BUNDLE DESTINATION .
    LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
)
```

OUTPUT:

Uploaded Folder Image.

33. Light Mode / Dark Mode:

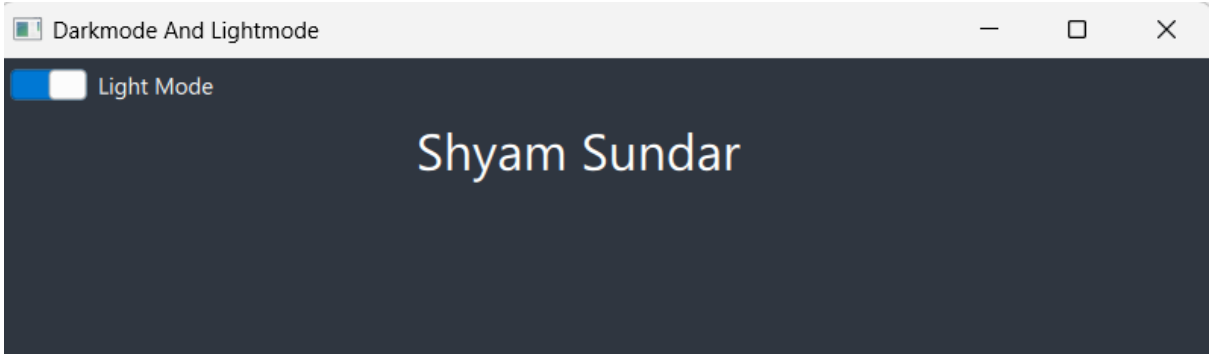
```
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts
```

```
Window {
    width: 640
    height: 480
    visible: true
    title: "Darkmode And Lightmode"

    AppTheme
    {
        id: appTheme
    }

    Component.onCompleted: appTheme.setLightTheme()

    Rectangle
    {
        width: parent.width
        height: parent.height
        color: appTheme.backgroundcolor

        Switch
        {
            id: themeSwitch
            text: "Light Mode"
            anchors.top: parent.top
            checked: false
            anchors.horizontalCenter: parent.hroizontalCenter

            contentItem: Text {
                text: themeSwitch.text
                color: appTheme.textcolor
                verticalAlignment:  Text.AlignVCenter
```

```
        leftPadding: themeSwitch.indicator.width + themeSwitch.spacing
    }

    onCheckedChanged:
    {
        if (checked)
        {
            appTheme.setDarkTheme()
            text: "Dark Mode"
        }
        else
        {
            appTheme.setLightTheme()
            text: "Light Mode"
        }
    }
}

ColumnLayout
{
    anchors.centerIn: parent
    Text {

        Layout.preferredWidth: 200
        Layout.preferredHeight: 100
        Layout.alignment: Qt.AlignHCenter
        id: myText
        text: qsTr("Shyam Sundar")
        color: appTheme.textcolor
        font.pointSize: 20
    }
```

```
        }
    }
}
```

OUTPUT:



34. Blend Mode Effect:

```
import QtQuick
import QtQuick.Window
import Qt5Compat.GraphicalEffects

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Item {
        anchors.centerIn: parent
        width: parent.width
        height: parent.height

        Image {
            id: nightBackground
            source: "https://images3.alphacoders.com/589/589870.jpg"
            sourceSize: Qt.size(parent.width, parent.height)
```

```
        smooth: true
        visible: false
    }

    Image {
        id: simbaLion
        source: "https://upload.wikimedia.org/wikipedia/en/9/94/Simba_%28_Disney_character_-_adult%29.png"
        sourceSize: Qt.size(parent.width, parent.height)
        smooth: true
        visible: false
    }

    Blend {
        anchors.fill: nightBackground
        source: nightBackground
        foregroundSource: simbaLion
        mode: "normal"
    }
  }
}
```

OUTPUT:

IMAGE DISPLAY

35. Blend Effect:

```
import QtQuick
import QtQuick.Window
import QtQuick.Effects

Window {
  width: 640
```

```
    height: 480
    visible: true
    title: "Blend Effect - Qt6"

    Item {
        anchors.fill: parent

        Image {
            id: nightBackground
            source: "https://images3.alphacoders.com/589/589870.jpg"
            anchors.fill: parent
            visible: false
        }

        Image {
            id: simbaLion
            source: "https://upload.wikimedia.org/wikipedia/en/9/94/Simba_%28_Disney_character_-
_adult%29.png"
            anchors.fill: parent
            visible: false
        }

        MultiEffect {
            anchors.fill: parent
            source: simbaLion        // foreground
            maskSource: nightBackground   // background
            maskEnabled: true
            maskThresholdMin: 0.1
        }
    }
}
```

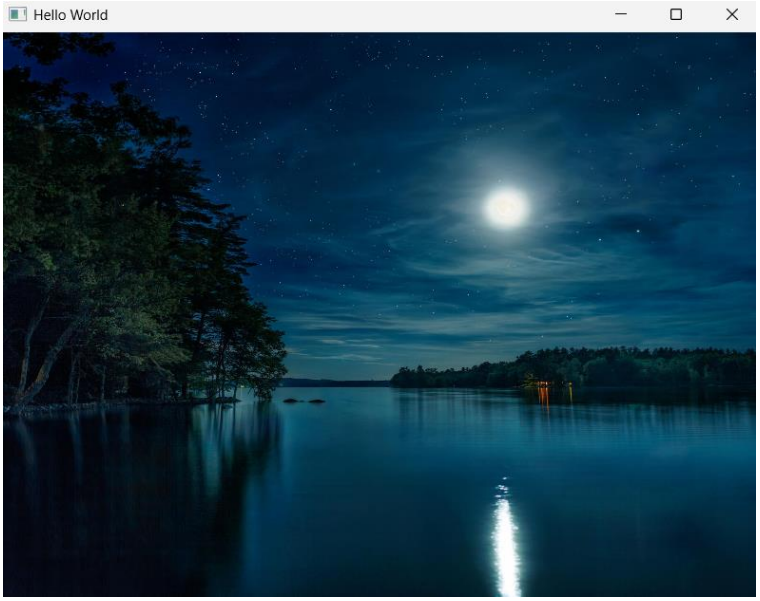
OUTPUT:

36. Gamma Adjust:

```
import QtQuick
import QtQuick.Window
import QtQuick.Effects   // for future effects, not used here

Window {
    width: 640
    height: 480
    visible: true
    title: qsTr("Hello World")

    Item {
        anchors.fill: parent

        Image {
            id: sourceImage
            anchors.fill: parent
            source: "https://images3.alphacoders.com/589/589870.jpg"
            fillMode: Image.PreserveAspectCrop
```

```
        }

        ShaderEffect {
            anchors.fill: parent
            property real gamma: 1.0
            property variant src: sourceImage

            fragmentShader: "
                uniform sampler2D src;
                uniform float gamma;
                varying vec2 qt_TexCoord0;
                void main() {
                    vec4 color = texture2D(src, qt_TexCoord0);
                    color.rgb = pow(color.rgb, vec3(1.0 / gamma));
                    gl_FragColor = color;
                }
            "
        }
    }
}
```

OUTPUT:



37. QT Image Zoom Gallery:

```
import QtQuick
import QtQuick.Window
import QtQuick.Controls

Window {
    width: 900
    height: 600
    visible: true
    title: "Pinch Zoom + Next/Prev Gallery"

    // ------- Image List --------
    property var images: [
        "https://picsum.photos/id/1011/800/600",
        "https://picsum.photos/id/1025/800/600",
        "https://picsum.photos/id/1033/800/600",
        "https://picsum.photos/id/1050/800/600"
    ]
    property int index: 0
```

```
Rectangle {
    anchors.fill: parent
    color: "black"

    Flickable {
        id: flick
        anchors.fill: parent
        clip: true
        contentWidth: img.width * img.scale
        contentHeight: img.height * img.scale

        Image {
            id: img
            source: images[index]
            fillMode: Image.PreserveAspectFit
            smooth: true
            transformOrigin: Item.Center

            // ---- PINCH ZOOM ----
            PinchHandler {
                id: pinch
                target: img
                minimumScale: 1.0
                maximumScale: 5.0

                onScaleChanged: {
                    flick.contentWidth = img.width * img.scale
                    flick.contentHeight = img.height * img.scale
                }
            }
```

```
        // ---- DRAG PAN ----
        DragHandler {
            id: dragger
            target: img
        }

        // ---- DOUBLE TAP RESET ----
        TapHandler {
            onDoubleTapped: resetView()
        }
    }
}

// ---- PREVIOUS BUTTON ----
Button {
    text: "<"
    anchors.left: parent.left
    anchors.verticalCenter: parent.verticalCenter
    anchors.margins: 20
    width: 50; height: 50
    font.pixelSize: 22

    enabled: index > 0
    onClicked: {
        index--
        resetView()
    }
}

// ---- NEXT BUTTON ----
```

```
        Button {
            text: ">"
            anchors.right: parent.right
            anchors.verticalCenter: parent.verticalCenter
            anchors.margins: 20
            width: 50; height: 50
            font.pixelSize: 22

            enabled: index < images.length - 1
            onClicked: {
                index++
                resetView()
            }
        }
    }

    // ---- Reset Zoom + Pan Function ----
    function resetView() {
        img.scale = 1.0
        flick.contentX = 0
        flick.contentY = 0
    }
}
```

OUTPUT:

38. Qt QML Navigation Drawer Slide Animation:

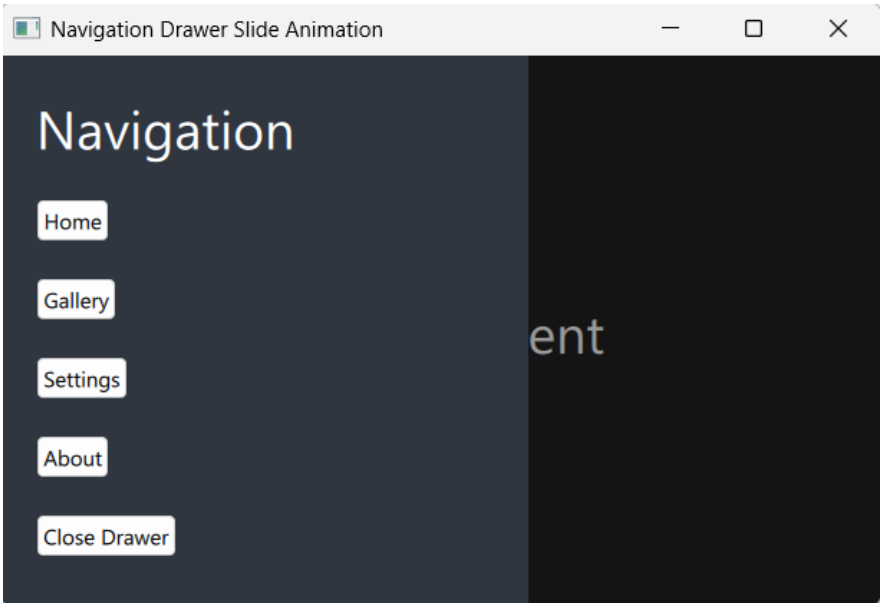
```
import QtQuick
import QtQuick.Window
import QtQuick.Controls

Window {
    width: 800
    height: 600
    visible: true
    title: "Navigation Drawer Slide Animation"

    property bool drawerOpen: false

    // -------- BACKGROUND CONTENT --------
    Rectangle {
        anchors.fill: parent
        color: "#222"

        Text {
            text: "Main Content"
            color: "white"
            anchors.centerIn: parent
```

```
        font.pixelSize: 30
    }

    Button {
        text: "☰ Menu"
        anchors.left: parent.left
        anchors.top: parent.top
        anchors.margins: 20

        onClicked: drawerOpen = true
    }
}

// -------- DIM BACKGROUND WHEN OPEN --------
Rectangle {
    id: dimmer
    anchors.fill: parent
    color: "#000000"
    opacity: drawerOpen ? 0.4 : 0.0
    visible: opacity > 0
    Behavior on opacity { NumberAnimation { duration: 250 } }

    MouseArea {
        anchors.fill: parent
        enabled: drawerOpen
        onClicked: drawerOpen = false
    }
}

// -------- NAVIGATION DRAWER --------
Rectangle {
```

```
id: drawer
width: parent.width * 0.6
height: parent.height
color: "#2f3640"
y: 0

// Slide Animation
x: drawerOpen ? 0 : -width
Behavior on x {
    NumberAnimation {
        duration: 300
        easing.type: Easing.InOutQuad
    }
}

Column {
    anchors.fill: parent
    spacing: 20
    padding: 20

    Text {
        text: "Navigation"
        color: "white"
        font.pixelSize: 30
    }

    Button { text: "Home" }
    Button { text: "Gallery" }
    Button { text: "Settings" }
    Button { text: "About" }
```

```
        Button {
            text: "Close Drawer"
            onClicked: drawerOpen = false
        }
    }
  }
}
```

OUTPUT:



39. QT QML CARD FLIPPING ANIMATION:

<u>Cardflip.qml:</u>

```
import QtQuick
import QtQuick.Controls
import QtQuick.Layouts

Item {
  id: cardItem
  width: 200
  height: 200
```

```
property bool isFlippingRunning: false
property bool isflipped: false
property int lastSuitIndex: 0
property int currentSuitIndex: 0

function currentSuit(index) {
    switch (index) {
    case 0: return "\u2660"   // Spade
    case 1: return "\u2665"   // Heart
    case 2: return "\u2666"   // Diamond
    case 3: return "\u2663"   // Club
    default: return ""
    }
}

function currentSuitColor(index) {
    switch (index) {
    case 0: return "000000"
    case 1: return "FF0000"
    case 2: return "FF0000"
    case 3: return "000000"
    default: return ""
    }
}

Rectangle {
    id: card
    width: parent.width
    height: parent.height
    radius: 10
```

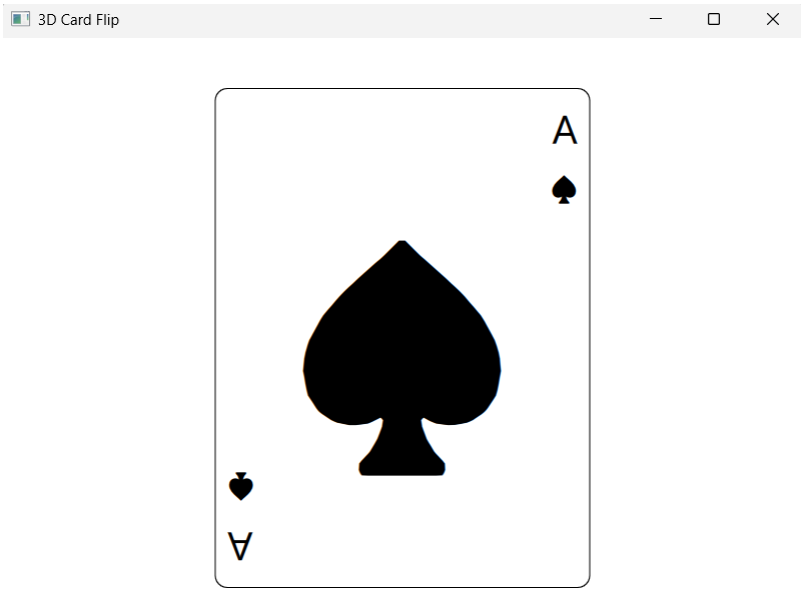
```
transform: Rotation {
    id: yRotation
    origin.x: card.width / 2
    origin.y: card.height / 2
    axis { x: 0; y: 1; z: 0 }
    angle: 0
}

Item {
    id: cardDesign
    anchors.fill: parent
    visible: true

    Rectangle {
        anchors.fill: parent
        color: "white"
        border.color: "black"
        radius: 10
    }

    ColumnLayout {
        anchors.left: parent.left
        anchors.top: parent.top
        anchors.margins: 10

        Text {
            text: "A"
            font.pointSize: 24
            color: "black"
        }
```

```
    Text {
        text: currentSuit(currentSuitIndex)
        font.pointSize: 24
        color: "black"
    }
}

Text {
    anchors.centerIn: parent
    text: currentSuit(currentSuitIndex)
    font.pointSize: 200
    color: "black"
}

ColumnLayout {
    anchors.bottom: parent.bottom
    anchors.right: parent.right
    anchors.margins: 10
    rotation: 180
    transformOrigin: Item.Center

    Text {
        text: "A"
        font.pointSize: 24
        color: "black"
    }

    Text {
        text: currentSuit(currentSuitIndex)
        font.pointSize: 24
        color: "black"
```

```
        }
    }
}

// ----- FLIP STATE -----
states: State {
    name: "flipped"
    when: cardItem.isflipped

    PropertyChanges {
        target: yRotation
        angle: 180
    }
}

// ----- TRANSITION -----
transitions: Transition {
    NumberAnimation {
        id: animFlip
        target: yRotation
        property: "angle"
        duration: 1000
        easing.type: Easing.InOutQuad

        onRunningChanged: {
            if (!running) {
                cardItem.isflipped = false
                cardItem.isFlippingRunning = false
                yRotation.angle = 0
            }
        }
```

```
            }
        }
    }

    // ----- MOUSE CLICK -----
    MouseArea {
        anchors.fill: parent

        onClicked: {
            lastSuitIndex = currentSuitIndex
            currentSuitIndex = currentSuitIndex + 1

            if (currentSuitIndex >= 4)
                currentSuitIndex = 0

            cardItem.isflipped = true
            cardItem.isFlippingRunning = true
        }
    }
}
```

OUTPUT:



40. Qt QML Animated Progress Bar:

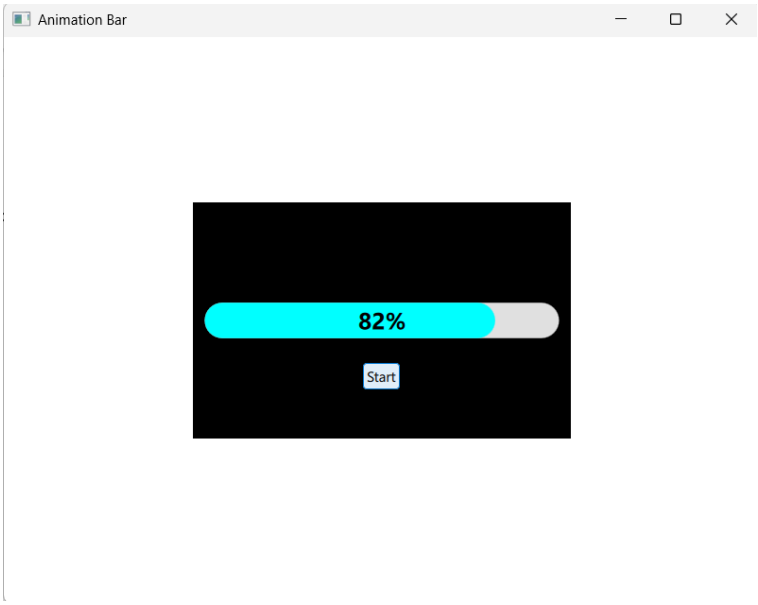
```
import QtQuick
import QtQuick.Window
import QtQuick.Controls
import QtQuick.Layouts
import QtQuick.Effects

Window {
    width: 640
    height: 480
    visible: true
    title: "Animation Bar"

    Rectangle {
        anchors.centerIn: parent
        width: 320
        height: 200
        color: "black"
```

```
Rectangle {
    id: progressBackground
    width: 300
    height: 30
    anchors.centerIn: parent
    color: "#e0e0e0"
    radius: 15

    Rectangle {
        id: progressBar
        width: 0
        height: parent.height
        radius: 15
        color: "#00FFFF"
        smooth: true
    }

    Text {
        anchors.centerIn: parent
        text: Math.round((progressBar.width / progressBackground.width) * 100) + "%"
        font.pointSize: 15
        font.bold: true
        color: "black"
    }

    /* ---------- Glow Replacement ----------- */
    MultiEffect {
        id: glowEffect
        anchors.fill: progressBar
        source: progressBar
        blurEnabled: true
```

```
        blur: 0.0               // will animate
        blurMax: 40             // max glow intensity
        brightness: 0.3
        saturation: 1.2
        visible: true

        Behavior on blur {
            NumberAnimation {
                duration: 500
                easing.type: Easing.InOutQuad
            }
        }
    }
}

Button {
    text: "Start"
    anchors.horizontalCenter: parent.horizontalCenter
    anchors.top: progressBackground.bottom
    anchors.topMargin: 20

    onClicked: {
        progressBar.width = 0
        progressAnim.start()
    }
}

ParallelAnimation {
    id: progressAnim

    NumberAnimation {
```

```
            target: progressBar
            property: "width"
            from: 0
            to: 300
            duration: 2000
            easing.type: Easing.OutBounce
        }

        NumberAnimation {
            target: glowEffect
            property: "blur"
            from: 0
            to: 20      // glow intensity
            duration: 800
            easing.type: Easing.InOutQuad
        }
    }
  }
}
```

OUTPUT:



41. Qt QML Slide:

PageTransitionsDemo/

- **main.qml**
- **SlidePage.qml**
- **FadePage.qml**
- **ScalePage.qml**
- **CubePage.qml**
- **ParallaxPage.qml**
- **PageManager.qml**

Main.qml:

```
import QtQuick

import QtQuick.Controls

import QtQuick.Window


Window {

   width: 640

   height: 480

   visible: true

   title: "QML Page Transition Demo"


   PageManager {

      id: pageManager

      anchors.fill: parent

   }

}
```

PageManager.qml:

```
import QtQuick

import QtQuick.Controls


Item {

   id: root
```

```
anchors.fill: parent

property var pages: [
    "SlidePage.qml",
    "FadePage.qml",
    "ScalePage.qml",
    "CubePage.qml",
    "ParallaxPage.qml"
]

property int currentIndex: 0

Loader {
    id: pageLoader
    anchors.fill: parent
    source: pages[currentIndex]
}

// ----------- NEXT BUTTON -----------
Button {
    text: "Next"
    anchors.right: parent.right
    anchors.bottom: parent.bottom
    anchors.rightMargin: 20
    anchors.bottomMargin: 20

    onClicked: {
        currentIndex = (currentIndex + 1) % pages.length
        pageLoader.source = pages[currentIndex]
    }
}
```

```
    // ---------- TITLE DISPLAY ----------
    Rectangle {
        width: parent.width
        height: 40
        color: "#202020"
        opacity: 0.8

        Text {
            anchors.centerIn: parent
            text: pageLoader.source
            color: "white"
            font.bold: true
        }
    }
}
```

SlidePage.qml:

```
import QtQuick
import QtQuick.Controls

Rectangle {
    width: parent.width
    height: parent.height
    color: "#3498db"

    Text {
        text: "Slide Transition"
        anchors.centerIn: parent
        font.pixelSize: 28
        color: "white"
```

```
    }

    states: State {
        name: "entered"
        PropertyChanges { target: root; x: 0 }
    }

    transitions: Transition {
        NumberAnimation { properties: "x"; duration: 500; easing.type: Easing.OutQuad }
    }

    Component.onCompleted: {
        x = -root.width
        state = "entered"
    }
}
```

FadePage.qml:

```
import QtQuick

Rectangle {
    id: root
    anchors.fill: parent
    color: "#8e44ad"
    opacity: 0

    Text {
        anchors.centerIn: parent
        text: "Fade Transition"
        color: "white"
        font.pixelSize: 28
```

```
    }

    Behavior on opacity {
        NumberAnimation { duration: 600 }
    }

    Component.onCompleted: opacity = 1
}
```

ScalePage.qml:

```
import QtQuick

Rectangle {
    id: root
    anchors.fill: parent
    color: "#2ecc71"
    scale: 0.3
    opacity: 0

    Text {
        anchors.centerIn: parent
        text: "Scale Transition"
        font.pixelSize: 28
        color: "white"
    }

    Behavior on scale { NumberAnimation { duration: 400; easing.type: Easing.OutBack } }
    Behavior on opacity { NumberAnimation { duration: 400 } }

    Component.onCompleted: {
        scale = 1
```

```
        opacity = 1
    }
}
```

ParallaxPage.qml:

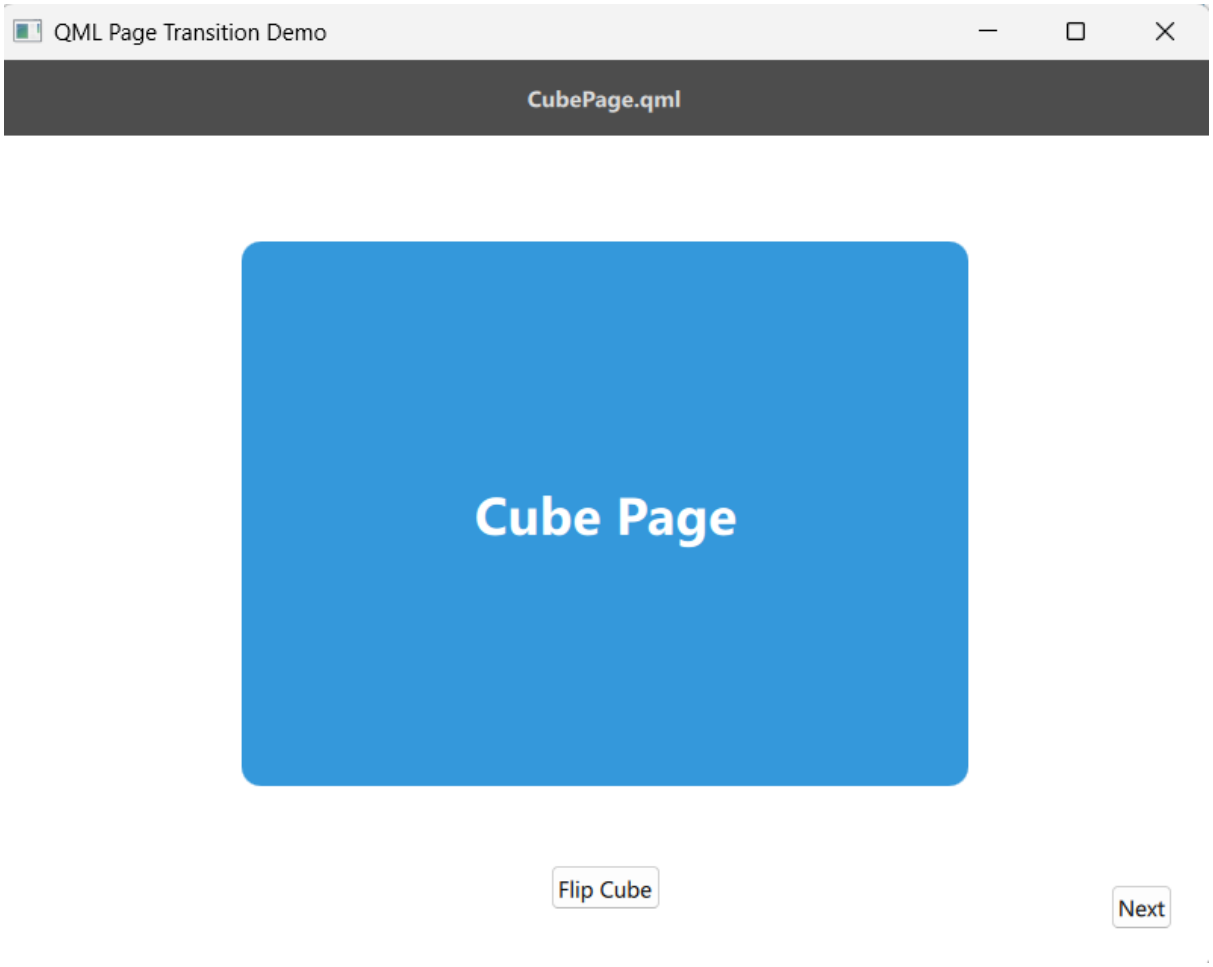
```
import QtQuick

Item {
    id: root
    anchors.fill: parent

    Rectangle {
        id: background
        anchors.fill: parent
        color: "#1abc9c"
        x: -parent.width
        Behavior on x { NumberAnimation { duration: 600 } }
    }

    Rectangle {
        id: foreground
        anchors.fill: parent
        color: "transparent"

        Text {
            anchors.centerIn: parent
            text: "Parallax Transition"
            font.pixelSize: 30
            color: "white"
        }
```

```
        x: parent.width
        Behavior on x { NumberAnimation { duration: 500; easing.type: Easing.OutQuad } }
    }

    Component.onCompleted: {
        background.x = 0
        foreground.x = 0
    }
}
```

CubePage.qml:

```
import QtQuick
import QtQuick.Controls

Item {
    id: page
    anchors.fill: parent
    property real angle: 0

    Rectangle {
        id: cubeFace
        width: parent.width * 0.6
        height: parent.height * 0.6
        anchors.centerIn: parent
        color: "#3498db"
        radius: 10

        transform: Rotation {
            id: rot
            origin.x: cubeFace.width / 2
            origin.y: cubeFace.height / 2
```

```
        axis { x: 0; y: 1; z: 0 }   // Y-axis rotation
        angle: page.angle
    }

    Text {
        anchors.centerIn: parent
        text: "Cube Page"
        color: "white"
        font.bold: true
        font.pixelSize: 28
    }
}

// BUTTON TO INITIATE ROTATION
Button {
    text: "Flip Cube"
    anchors.bottom: parent.bottom
    anchors.horizontalCenter: parent.horizontalCenter
    anchors.bottomMargin: 30

    onClicked: cubeFlipAnim.start()
}

// 3D CUBE ROTATION ANIMATION
NumberAnimation {
    id: cubeFlipAnim
    target: page
    property: "angle"
    from: 0
    to: -90
    duration: 700
```

```
        easing.type: Easing.InOutQuad
    }
}
```

OUTPUT: